MosaicOpt: An Algebraic Foundation for Agentic Memory Databases

Abstract

Production LLM agents accumulate multi-modal memory across two timescales: long-term facts and preferences, and short-term working context. A memory unit typically carries text, one or more dense vector embeddings, structured metadata, and graph relations. Useful queries against this store are conjunctive expressions that mix all four modalities, for example, "find passages about a fault-tolerance bug from the last 30 days, semantically similar to a stack trace, attributed to a known subsystem, and connected by a derivation link to a prior incident." The dominant engineering response, *multi-path recall* with application-level score fusion, has no principled semantics, no equivalence relation over plans, and breaks down on multi-vector conjunctions. We propose **MosaicOpt**, an *algebra-first* design that extends relational algebra with (i) a *materialization* operator μ that binds computed values (vector distances, BM25 scores) as first-class attributes, (ii) first-class vector, text, and graph predicates, and (iii) a graph expansion operator Γ. The optimizer is a Volcano/Cascades-style rule-based layer driven by an *equational theory* of nine foundational identities (M1–M6, G1–G3) over the extended operators, supplemented by the assumed classical A1, A4, A7, A8 (selection composition, selection–join pushdown, join commutativity and associativity; A1 and A4 apply naturally to vector predicates), the derived / definitional rewrites (A4-vector/J1, A7-symmetry, A8-regrouping, the τ-Λ-topK rewrite, and the row-preserving σ-τ commutation), combined with a cost-based layer that explicitly accounts for approximation. We prove the identities, give counterexamples for the non-identities (N1–N5) that classical intuition suggests but approximation, graph-traversal asymmetry, or the traversal-vs-origin column split break, and sketch a prototype and evaluation plan. The argument is foundational: agentic memory is a query problem, query problems benefit from algebraic foundations, and the right foundation is an extension of relational algebra.

ACM Reference Format:

. 2018. MosaicOpt: An Algebraic Foundation for Agentic Memory Databases. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 19 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction & Motivation

LLM agents accumulate state across two timescales: *long-term memory* (persistent facts, preferences, episodic traces) and *short-term memory* (working context of an in-flight session). A memory unit carries raw text, one or more dense vector embeddings, structured metadata, and graph relations (`derived_from`, `cites`, *etc.*). Useful queries are *conjunctive* across all four modalities, "find passages about a fault-tolerance bug from the last 30 days, semantically similar to a stack trace, attributed to a known subsystem, and linked by `derived_from` to a previous incident" is representative.

Running example: a real agentic-memory query

"Find ***fault-tolerance*** *passages from the* ***last 30 days***, ***body***- *and* ***summary***-*similar to a stack trace, attributed to a known* ***subsystem***, *linked by* `derived_from` *to a previous incident,* ***top 10*** *by* (d_1, d_2)*."*

$$
\begin{aligned}
Q_1 &= Mem \\
Q_2 &= \sigma_{tag=\text{ft},\ sub=\text{known},\ ts>\text{now}()-30d}(Q_1) \\
Q_3 &= \mu_{\|emb-stack\|\to d_1}\, \mu_{\|emb_alt-focus\|\to d_2}(Q_2) \\
Q_4 &= \sigma_{d_1<\tau_1\ \wedge\ d_2<\tau_2}(Q_3) \\
Q_5 &= \Gamma_{derived_from,\, d=1}(Q_4) \\
Q_6 &= \Lambda_{10}(\tau_{d_1,d_2}(Q_5))
\end{aligned}
$$

Figure 1: Running example: the same agentic-memory query as a natural-language ask and as an algebraic expression.

The dominant engineering response is *multi-path recall*: a fan-out across specialized engines. Frameworks such as Mem0 [23], Letta [19], Zep, and Cognee instantiate variations. The pattern has four structural weaknesses: (1) no principled semantics; two implementations of "find similar memory" can have silently different meanings; (2) heuristic optimization only; there is no equivalence relation over plans, and rewrites are not derived from a sound theory; (3) fusion policy (*e.g.*, `ORDER BY` $0.7d_1 + 0.3d_2$) leaks into the substrate, becoming hard to change without re-engineering the data path; and (4) per-column top-k breaks conjunctive intent because, for multi-vector memories, the per-column top-k sets are correlated, not redundant, and the intersection is frequently empty.

These problems are symptoms of a missing foundation. In this work, we present MosaicOpt[1], an *algebra-first* design: relational algebra extended with operators that make multi-modal memory queries first-class, with the algebra as the substrate contract. The optimizer is rule-based (Section 4) with a cost-based layer (Section 7) that may legitimately differ in approximation behavior.

Figure 1 instantiates the *application* layer (natural-language ask on the left, algebraic plan on the right). Beneath it sits the *logical algebra* (substrate contract, including new μ/Γ operators) and the *physical algebra* (concrete access paths: HNSW/IVF/flat, B-tree, inverted index, graph walks). ANN (approximate nearest neighbor) processing enters only at the physical layer via $H_{V,q,\tau}$; the cost model (Section 7) makes this explicit.

1.1 Why an algebraic foundation?

Codd's relational algebra [4], Selinger's optimizer [33], and the Cascades framework [7, 8] have already demonstrated that *An algebra is not an implementation detail; it is the contract between the query and execution.*

[1] A "mosaic" of text/embedding/structured-metadata/graph pieces fused in one logical algebra, turned into an executable plan by an RBO+CBO+cost-model "Opt".

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Given that agentic memory operates over two temporal scales (short- and long-term), exhibits intrinsically multimodal content, and supports productive queries (*e.g.,* episodic recall, cross-session fact retrieval, working-context assembly, and tool-trace analysis) that are *conjunctive across modalities*, an architecture based on a heterogeneous "bag of engines" is not methodologically appropriate. Frameworks such as Mem0 [23] (a memory layer that extracts salient facts from conversations and stores them in a vector index for later retrieval), Letta [19] (a stateful-agent framework that maintains per-agent memory blocks across sessions), Generative Agents [26] (stanford-style LLM agents with a memory stream), MemGPT [25] (LLM-as-OS-style hierarchical memory), and ReAct-style agents [40] have independently reinvented the same multi-path recall, score fusion, and re-ranking patterns. In spite of the engineering efforts, the semantic substrate is underspecified. Current frameworks expose *memory API*s without formal semantics, so the same SQL-like query may behave differently across implementations. As a result, migrating workloads often requires reading source code instead of reasoning from a stable specification. Mem0 models facts as a flat message sequence (optionally with embeddings), while Letta uses a typed block hierarchy split into archival and recall stores. Neither approach offers an explicit plan-equivalence relation for reasoning about interchangeable retrieval and single-path recall strategies.

In production, the four main agentic-memory use cases (*i.e.,* episodic recall, cross-session fact retrieval, working-context assembly, and tool-trace analysis), can all be expressed as conjunctive queries combining vector similarity, metadata predicates, and graph constraints over a single storage substrate, enabling a shared optimization pipeline. The proposed MosaicOpt meets these requirements by offering ❶ a formal, expressive language; ❷ an equational theory for semantics-preserving rewrites; ❸ a cost model parameterized by user-defined logical operators; and ❹ compositional mechanisms for adding operators.

1.2 Contributions

MosaicOpt sits at the intersection of three research threads: (1) the classical RDBMS literature on cost-based optimization and rule-based rewriting [3, 7–9, 11, 33]; (2) the modern vector-database literature on ANN search and the approximation-quality trade-off [6, 10, 12, 13, 15, 20–22, 24, 27, 32, 34, 35]; and (3) the Agentic-memory literature on multi-path recall and memory stores for LLM agents [19, 23, 25, 26, 40]. MosaicOpt introduces an algebraic foundation that lifts the Agentic-memory workload from a hand-tuned multi-path orchestrator to a first-class optimizer target, with the approximation budget and the equational theory as the contract between the application code and the engine.

Overall, this paper makes the following contributions:

- We introduce an extended relational algebra with materialization μ and graph expansion Γ operators and first-class vector/text/graph predicates.
- Based on the newly added algebraic operators, we present the corresponding equational theory: nine foundational identities and five non-identities with proofs in Appendix B.
- We introduce a three-layer architecture that separates SQL fusion policy, logical algebra, and physical algebra.

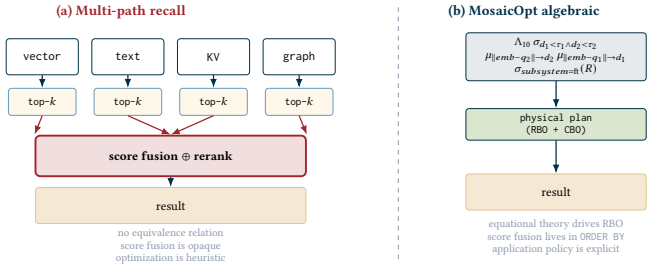


Figure 2: Multi-path recall (a) vs. MosaicOpt algebraic (b).

- A cross-modal plan enumeration that yields a small (2–4 plans) but complete plan space via modality-aware pruning is introduced in MosaicOpt.
- We implement the prototype on both DuckDB and PostgreSQL, and evaluate the performance on three popular datasets.

The proposed system contributions elevate the algebraic formulation from a purely notational device to an executable query optimizer. In addition, MosaicOpt delineates a promising approach for supporting general FANN (filtered ANN) [32] queries involving multiple attributes and multiple vector columns.

2 Related work

Codd's relational algebra [4] and Selinger's optimizer [33] showed that effective query processing requires (i) a formal query language and (ii) a theory of semantics-preserving rewrites. Cascades [7, 8] systematized this as a *rule-based* control loop over a *cost-based* search, with equivalence rules as first-class objects. MosaicOpt follows this lineage, grounded in an *equational theory* over plans and *logical–physical separation*.

Modern IR extends classical ranking, *e.g.,* the vector space model [31] and BM25 variants [29, 30], with dense embeddings (word2vec [24], GloVe [27], Transformers [35], BERT [5], Sentence-BERT [28]). HNSW [21] is a dominant ANN index, with recall–latency governed by ef and M. Although approximation is often hidden at the algebraic level, it is central to cost modeling: higher recall costs more, so the optimizer must manage this trade-off.

Hybrid systems largely fall into three groups: (1) vector-aware extensions to relational DBMSs (pgvector, DuckDB `vss`, Oracle AI Vector Search) that add vector types and ANN indexes via functions; (2) vector-centric hybrid stores (Milvus [36], SingleStore-V [37], Manu [14], Pinecone, Weaviate, Qdrant, Vespa) with metadata filtering layered on; and (3) multi-modal query languages (SQL/MM, SPARQL, CQL) that mix graph/text/vector primitives but lack a unified equational basis for rewrites. A common limitation is that score fusion remains in the application layer instead of being encoded in the algebra that defines query answers.

Figure 2 delineates two alternative architectures: (a) four independent channels are provided as inputs to a fusion component in the absence of any defined equivalence relation; (b) the user specifies a single logical plan, the RBO systematically enumerates equivalent rewrites under the governing equational theory, the CBO selects an appropriate physical implementation, and the fusion policy is made explicit via `ORDER BY`. This comparison is salient along three dimensions. First, *equivalence*: configuration (a) lacks a

Table 1: Logical operators of the extended algebra. (In a table R, each attribute A, the entry of which should remain atomic logically, can be either scalar values, vectors, or texts.)

Operator	Syntax	Meaning
Materialization	$\mu_{e\to c}(R)$	Per row, compute e; bind to c.
Approximate Join	$R \bowtie_{distance(a,b)\le\tau} S$	approximate join between R and S.
Graph expansion	$\Gamma_{g,d}(R)$	From node-id, traverse g to depth d.

formal notion of equivalence, whereas (b) is grounded in an explicit equational theory. Second, *rewrites*: configuration (a) corresponds to a manually engineered orchestration pipeline, whereas (b) yields rewrites produced by the rule-based optimizer. Third, *extension*: configuration (a) necessitates modifications to pipeline code whenever a new modality or fusion policy is introduced, whereas (b) requires only the addition of the corresponding operator and its algebraic identities to the underlying algebra.

3 The Extended Algebra: Syntax & Semantics

We extend the classical relational algebra with four new constructs: a *materialization* operator μ that lifts computed values (vector distances, BM25 scores, derived attributes) to first-class attributes; first-class multi-modal predicates (vector range, BM25 relevance, graph reachability); kNN join over a pair of vectorized columns; and a *graph expansion* operator Γ. Bag semantics apply throughout (matching SQL, avoiding row-collapse anomalies of set semantics with duplicate memory units).

The full operator inventory:

Graph modality as a first-class edge relation. The graph modality is realized through a *distinguished first-class edge relation KG*(*src*, *dst*, *label*) that lives in the schema alongside *Mem* and any other user-declared relations. Γ's subscript g in $\Gamma_{g,d}(L)$ is the *edge label filter* on *KG.label*, so $\Gamma_{\text{derived_from},\,d=1}(Q_4)$ means: take each row of Q_4 as an origin, walk *KG* restricted to edges with *label* = `derived_from`, and return the rows reachable within one hop, joined with Q_4's other attributes. Γ does not need a second argument: the graph is a system-level resource like the base relation *Mem*, and the subscript g selects which edges within that resource to follow. The mapping from graph modality to the algebra is therefore one-to-one: *Mem* carries the memory-unit rows, *KG* carries the edges, and Γ is the operator that closes the loop.

Existing semantics: a relation is a multiset; σ, μ, Γ preserve row multiplicity; $\cup$ adds, $\cap$ takes min, $-$ subtracts; τ_o orders (subscript o names the sort column list, *e.g.,* τ_{d_1,d_2} sorts by (d_1, d_2)); Λ_k truncates (subscript k is the row count, *e.g.,* Λ_{10} keeps the top 10); Γ_g expands *KG* along edges of label g.

A predicate φ is a Boolean combination of atoms over the input attributes: *classical* ($A = c$, $A < c$, $A \in S$). We extend it by enabling the predicate to involve vector distance, text similarity, and graph reachability tests, *i.e.,* $\|V_a - q\| < \tau$ for query-time bound q, threshold τ, $\text{BM25}(t, q) > \theta$, and $\text{reachable}(u, v)$, $\text{edge}(u, v, \ell)$. Specifically, we list the newly introduced predicates as follows,

- *vector*: $\|V_a - q\| < \tau$, where V_a is a vector-valued attribute, q is a query-time bound parameter, and $\tau \ge 0$ is a distance threshold; equality and other comparison operators are admissible, range ($< \tau$) is the most important form;
- *text*: $\text{BM25}(t, q) > \theta$, where t is a text attribute, q a query-time bound string, and θ a relevance threshold; we use Okapi BM25 with parameters (k_1, b) carried with the predicate;
- *graph*: $\text{reachable}(u, v)$ boolean and $\text{edge}(u, v, \ell)$ for nodes u, v in a graph attribute, ℓ a label or wildcard.

The query-time bound parameters (q, τ, θ) are first-class values supplied at execution time, distinct from input attributes: they make predicates safely late-binding. The algebra expression is the same across query instances; only the bound parameters change.

A materialization expression e is evaluated per tuple. Supported forms include: (i) scalar expressions (*e.g.,* $A + B$, $f(A)$); (ii) vector expressions (*e.g.,* $\|V_a - q\|$); (iii) text-retrieval expressions (*e.g.,* $\text{BM25}(t, q)$); (iv) Boolean/arithmetic compositions of the above; and (v) constants true, false, and null. The expression language is intentionally limited to user-visible SQL/MM-style predicates and to forms with estimable costs in the model.

The key property of μ is that it promotes the computed value to a first-class output attribute. After materialization, the value can be referenced by downstream selection σ, projection π, `ORDER BY` within sorting τ, and later μ expressions. This lets the system expose per-column distances to application `ORDER BY` semantics without dropping them during execution.

In $\mu_{e\to c}(L)$, the requirement that c be fresh (*i.e.,* absent from the input schema) functions as a hygienic condition to avoid name capture, rather than constituting a semantic constraint. When necessary, a renaming operation ρ may be applied to generate an appropriate fresh identifier.

The denotational semantics of the μ operator is given by the multiset $\{\, r \oplus \{c \mapsto e(r)\} : r \in R \,\}$, where $\oplus$ extends each tuple r with a binding that assigns the fresh attribute c the value obtained by evaluating e on r. If $e(r)$ is undefined (*e.g.,* when computing a distance to a null vector), the corresponding tuple is omitted from the result. This behavior is consistent with SQL `NULL`-propagation conventions and is required to ensure safety in the presence of missing embeddings.

Why not the classical assignment/extend operator? The extended-relational-algebra literature has long offered an *assignment* or *extend* operator, *i.e.,* $R.a := E$ or $R \oplus \{a \mapsto E\}$, that re-binds (or extends) one attribute of R in place [1]. μ differs in a single, decisive way: it is the optimizer's *first-class surface* for per-row multi-modal evaluation. The per-row expression e, the named binding c, the cost dimension T_μ (per-row evaluation cost, HNSW-bound for vector distances), and the M1–M5 redex patterns are all anchored on the μ node type. Two examples make the difference concrete. *(i) Cost pricing.* Take $Q = \sigma_{d<\tau}\big(\mu_{\|emb-q\|\to d}(R)\big)$ with $n = |R| = 10^4$, $d = 128$, and selectivity $s \approx 10^{-3}$. With μ, the cost model sees a μ node and prices it as an ANN-bound $T_\mu \approx 0.5\,\text{ms}$ (HNSW `efSearch`= 64), so the CBO correctly picks HNSW. An equivalent plan written as $\sigma_{d<\tau}\big(\pi_{\ldots,\|emb-q\|}(R)\big)$ exposes only a generic projection node, and the cost model prices T_π an order of magnitude lower, so the CBO wrongly picks a flat scan. *(ii) Rewriter anchoring.* The RBO's M1 redex $\sigma_{c<\tau}\circ\mu_{e\to c}$ is anchored on a μ node; rewriting it to $\mu_{e\to c}\circ\sigma_{e<\tau}$ saves a $(1-s)\cdot n$ factor of per-row distance computation. With a π-based encoding, M1 has no node type to match and does not fire, so the distance is computed for every row regardless of selectivity—at $s = 10^{-3}$, that is a 1000× wasted work. Classical assignment is a

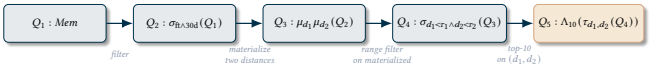


Figure 3: The running example $Q_1 \rightarrow Q_5$ as a horizontal pipeline; the warm-tan terminal box is the top-K over the materialised distance columns.

general-purpose extension that is not anchored on a rewriter or a cost entry; the multi-modal workload would lose both.

The approximate join operator, $R \bowtie_{dist(a,b)\leq\tau} S$, performs a θ join over R and S, where θ is extended to involve distance filters over vectors and text columns. Specifically, for each tuple in R, it joins with those tuples in S whose entries in column $S.b$ are within (or beyond) the distance predicate τ, with respect to the entry of $R.a$. Hereby, the distance ($dist(a, b)$) can be either $\texttt{BM25}(a, b)$ or $||a - b||$.

When is this operator forced? The "approximate" qualifier in the operator's name refers to the *physical* implementation, not the logical semantics: the logical operator computes the exact θ-join on the distance predicate, but the cost model admits an ANN-backed physical realization (an HNSW traversal of the inner relation) since the cross-table θ-join over a distance threshold has no classical B-tree acceleration. This operator is forced by three workload shapes for which no other relational construct suffices. *(i) Cross-table semantic join, e.g.,* "find `Passages` with description similar to `Incidents.description`", joining two relations that share no key; classical equi-join infrastructure does not apply, and a nested-loop μ over the inner relation is asymptotically worse than the ANN-walked join this operator licenses. *(ii) Near-duplicate detection*, $R \bowtie_{\|a-b\|\leq\tau} R$, the canonical de-duplication pattern where the join key is a vector distance. *(iii) Entity matching over heterogeneous embeddings*, joining users to sessions by content vector similarity when the two sides are not even in the same relation schema. In all three cases the join key is a vector (or BM25) distance, so the operator exists exactly to give that pattern a first-class place in the algebra.

Running example. Let *Mem* be a memory-unit relation with attributes *id*, *text*, *ts*, ***emb***, ***emb_alt***, *user*, *subsystem*, and let $KG(src, dst, label)$ hold derivation links. The query "semantically similar passages in the fault-tolerance subsystem from the last 30 days, top 10 by (d_1, d_2)" is:

$$
\begin{aligned}
Q_1 &= Mem \\
Q_2 &= \sigma_{subsystem=\text{ft}\wedge ts>\text{now}-30d}(Q_1) \\
Q_3 &= \mu_{\|\boldsymbol{emb}-\boldsymbol{q}_1\|\rightarrow d_1}\ \mu_{\|\boldsymbol{emb_alt}-\boldsymbol{q}_2\|\rightarrow d_2}(Q_2) \\
Q_4 &= \sigma_{d_1<\tau_1\wedge d_2<\tau_2}(Q_3) \\
Q_5 &= \Lambda_{10}(\tau_{d_1,d_2}(Q_4))
\end{aligned}
$$

Figure 3 shows the same example as a horizontal pipeline. Each box is a logical operation, emphasizing compositionality: each Q_i can be referenced as input to the next stage.

A graph-aware variant (*i.e.,* "the same computation, restricted to memories connected via a `derived_from` chain to a known incident"? can be formalized as

$$
\begin{aligned}
R_1 &= \sigma_{id=42}(Mem) && \text{— the seed incident} \\
R_2 &= \Gamma_{\text{derived_from},\, d=2}(R_1) && \text{— two hops downstream} \\
R_3 &= \sigma_{id\in\pi_{dst}(R_2)}(Q_2) && \text{— intersect with filter}
\end{aligned}
$$

followed by the same $\mu/\sigma/\tau/\Lambda$ sequence. Here, Γ composes with the rest of the algebra. This example shows that (i) the algebra is closed under composition (each operator outputs a logical plan usable as input to any other operator); and (ii) graph expansion composes with the algebra like selection, projection, and join.

4 Equational Theory: Identities and Non-Identities

4.1 Why these operators?

Why σ, π, $\cup$, $-$, ρ, $\times$, τ, Λ? They are the classical relational algebra; we do not add or remove any of them. Why μ (materialization) and Γ (graph expansion)? They are the two operators needed to express the Agentic-memory workload in the algebraic form, every memory query either materializes a computed value (a distance, a relevance score, a derived attribute) or expands the graph (traverses a derivation chain). The fusion pattern $\mu_{d_a}\ \mu_{d_b}\ \sigma_{d_a<\tau_a\wedge d_b<\tau_b}\ \tau_{d_a+d_b}\ \Lambda_k$ from Section 3 is the canonical multi-modal fusion; the $\sigma_\varphi\ \mu_{e\rightarrow c}\ \Gamma_{g,d}$ chain is the canonical graph-aware materialization. The choice of these two operators is the minimum that makes the Agentic-memory workload expressible; adding more operators would only complicate the rewrite theory without expanding the expressive power. Operators we did *not* add: a path operator (expressible as μ + Γ), a streaming operator (orthogonal to the algebra), a probabilistic operator (future work, see Section G.2).

The optimizer (Section 6) is driven by an *equational theory*: a set of identities over logical plans. The same theory also records, explicitly, the identities that classical intuition suggests but approximation breaks, the *non-identities*, so the optimizer knows not to apply them. This section unifies both sides. We first give an overview (4.1), then the inherited classical axioms (4.2), then the new identities that the extended operators license (4.3), then the derived rewrites the optimizer uses (4.4), and finally the non-identities with counterexamples (4.5).

4.2 Overview

The equational theory of the extended algebra plays two complementary roles. *First*, it is the foundation for the rule-based optimizer (Section 6): every rewrite the RBO performs is justified by a foundational identity (M1–M5, G1, G2) or by a derived corollary / definitional / implied rule (A4-vector, A4-vector+M1, M4+M1, M5+G1, A7-symmetry, A8-regroup, topK recognition, σ-τ commutation), so the optimizer's behaviour is auditable, not opaque. *Second*, the *non*-identities (N1–N5) are the discipline that prevents the optimizer from applying rewrites that hold in classical relational algebra but break under approximation, graph-traversal asymmetry, or the traversal-vs-origin column split. The cost model (Section 7) is responsible for the physical cost of every rewrite; the equational theory is responsible for its *correctness*.

Concretely, an identity is an equation of the form `LHS = RHS` between two logical plans that hold for all instances of the free variables (relation names, attribute names, query-time bound parameters). A non-identity is a pair of plans that classical intuition might equate but the extended setting does not, with an explicit counterexample. Both are first-class objects: the RBO loads them as data, the cost model consults them when computing the "cost of identity application" term, and the executor relies on them for safety of adaptive re-planning (Section 7).

The structure of this section follows the algebra's structure: inherited axioms (the ones we use but do not re-prove) in Section 4.3; new identities (the ones the extended operators license) in Section 4.4; derived rules (combinations used directly by the optimizer) in Section 4.5; and non-identities (counterexamples that bound the optimization horizon) in Section 4.6.

4.3 Inherited foundational axioms

The following four identities are the assumed classical axioms the new rules actually depend on. Each is named because the rewriter in Section 6 matches it explicitly. Other standard classical identities (selection over union, projection idempotence, projection–join pushdown, rename involutivity, etc.) hold throughout the paper and are taken as implicitly assumed; we omit them because no rule in the new equational theory anchors on them.[2]

- **(A1) Selection composition.** $\sigma_\varphi \sigma_\psi(L) = \sigma_{\varphi \wedge \psi}(L)$. Used by the rewriter to split a conjunctive σ into per-predicate selections, so each can be pushed individually (see M1, M6, A4-vector, V1).
- **(A4) Selection–join pushdown.** If every attribute mentioned in φ is in the schema of L, then $\sigma_\varphi(L \bowtie_\psi M) = \sigma_\varphi(L) \bowtie_\psi M$. Used by the rewriter to push predicates that reference only one side of a join (M4 and its corollary A4-vector).
- **(A7) Join commutativity.** For any symmetric predicate ψ, $L \bowtie_\psi M = M \bowtie_{\psi^{\text{sym}}} L$. The two sides are identical as multisets; the cost is not (Theorem 12). The RBO generates both LHS and RHS orderings; the CBO picks the cheaper side.
- **(A8) Join associativity.** For independent predicates ψ, χ with no attribute ambiguity, $L \bowtie_\psi (M \bowtie_\chi N) = (L \bowtie_\psi M) \bowtie_\chi N$. This is the identity the RBO uses to enumerate the $O(n!)$ join orderings.

These hold for any predicate in our extended form (classical, vector, text, graph) provided the attribute references are respected. We do not claim any new identities here; we merely note that the extended predicates do not violate them.

4.4 New foundational axioms

We introduce the *nine* identities that the extended algebra makes possible. Each is stated in a form that names the conditions for its applicability. (Two further rules, **V1** and **J1**, are corollaries of the assumed classical axioms A1 and A4 with vector-predicate specialization; they are kept as named redex patterns in the RBO code but have no separate theorem block. The τ-Λ-topK family and the σ-τ commutation are *implied* by the operator definitions: $\text{topK}_O(L, K) := \tau_o(\Lambda_k(L))$ from Section 3 and the row-preserving semantics of σ and τ. The RBO uses them as zero-weight rewrites in the ARS, but they have no separate theorem block.)

[2]Concretely, A2, A3, A5, A6 are not referenced by any identity M1–M6, G1–G3, or by the rewrites they license. They are standard textbook axioms and are part of the implicit closed-world assumption on the classical algebra.

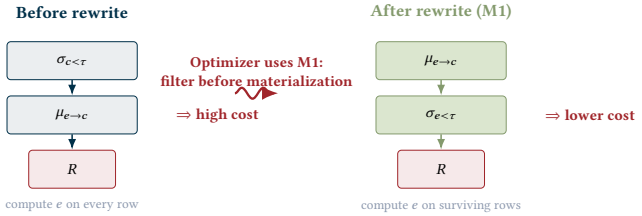


Figure 4: Visualization of the M1 rewrite: the optimizer swaps σ and μ so that e is computed only on rows that survive the filter. The optimized side is rendered in the soft-olive accent to make the rewrite visible at a glance.

Each identity below is followed by a brief *cost-effect annotation* that names the optimization the rewrite enables; the RBO fires the rewrite as part of its greedy loop (Section 6.1).

Theorem 1 (M1: Selection commutes with materialization on the materialized column)**.** *Where c is a fresh attribute name and e is an expression depending only on attributes of L:*

$$\sigma_{c<\tau}\, \mu_{e\to c}(L) \;=\; \mu_{e\to c}\, \sigma_{e<\tau}(L).$$

(Proof in Appendix B.)

Figure 4 visualizes the M1 rewrite. The *left* tree shows the input plan: $\sigma_{c<\tau}$ on top of $\mu_{e\to c}(R)$, which materializes e on every row and then filters on $c < \tau$. The *right* tree shows the rewritten plan: $\mu_{e\to c}$ on top of $\sigma_{e<\tau}(R)$, which applies the filter *before* materialization; e is computed only on rows that survive the filter. The cost saving scales with the selectivity of $e < \tau$, for a tight vector range the selectivity can be one part in 10^3 or smaller, and the saving is correspondingly dramatic.

Theorem 2 (M2: Materialization is commutative)**.** *For fresh names c_1, c_2 and expressions e_1, e_2 depending only on L's attributes (and not on each other's outputs):*

$$\mu_{e_1\to c_1}\, \mu_{e_2\to c_2}(L) \;=\; \mu_{e_2\to c_2}\, \mu_{e_1\to c_1}(L).$$

(Proof in Appendix B.)

Theorem 3 (M3: Materialization through projection)**.** *For fresh c and expression e depending only on attributes in A:*

$$\mu_{e\to c}\, \pi_A(L) \;=\; \pi_{A\cup\{c\}}\, \mu_{e\to c}(L).$$

(Proof in Appendix B.)

Theorem 4 (M4: Materialization pushdown through join)**.** *Where e depends only on attributes of L and c is a fresh attribute name:*

$$\mu_{e\to c}\, (L \bowtie_\psi M) \;=\; (\mu_{e\to c}(L)) \bowtie_\psi M.$$

(Proof in Appendix B.)

Theorem 5 (M5: Materialization pushdown through graph expansion)**.** *Where e depends only on attributes of L, c is fresh, and c is not the graph rule's node-id attribute:*

$$\mu_{e\to c}\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \mu_{e\to c}(L).$$

This is the μ-counterpart of G1 and the rewrite the RBO generates for graph queries that materialize a per-origin attribute (e.g. a distance to

a fixed query point) and then expand. The two sides produce the same multiset of (r, v, c) tuples; the cost differs because the LHS computes e on every traversal row while the RHS computes e on every origin row. When $|\Gamma_{g,d}(L)| > |L|$ (the typical case for $d \geq 1$ and a non-trivial graph), the RHS is cheaper, and the CBO picks it.

(Proof in Appendix B.)

Theorem 6 (M6: Selection on input attributes commutes with materialization)**.** *Where φ references only attributes of L (and not the fresh attribute c):*

$$\sigma_\varphi\, \mu_{e\to c}(L) \;=\; \mu_{e\to c}\, \sigma_\varphi(L).$$

Cost effect: *the LHS computes e on every row of L and then filters; the RHS filters first and computes e only on the s-fraction that survives. For selectivity s, the saving is $(1-s)\cdot|L|\cdot T_\mu$ per-row evaluations. M6 is the classical-filter complement of M1: M1 handles $\sigma_{c<\tau}\circ\mu_{e\to c}$ (predicates that reference c, the materialised attribute); M6 handles $\sigma_\varphi\circ\mu_{e\to c}$ (predicates that reference only the input schema, the typical case for $\sigma_{subsys=\text{known}}\circ\mu$ in the running example). Together they cover every σ–μ redex.*

(Proof in Appendix B.)

Theorem 7 (G1: Selection pushdown through graph expansion)**.** *Where φ references only attributes of L (and not attributes of the graph rule's traversal result):*

$$\sigma_\varphi\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \sigma_\varphi(L).$$

(Proof in Appendix B.)

Theorem 8 (G2: Graph expansion is transitive)**.** *Let g_1, g_2 be graph rules over the same graph attribute with disjoint label sets. If g_1's traversal-result attribute and g_2's source attribute are disjoint (no intermediate node of g_2 is a valid origin of g_1), then the composition $g_1; g_2$ satisfies:*

$$\Gamma_{g_1,d_1}\, \Gamma_{g_2,d_2}(L) \;=\; \Gamma_{g_1;g_2,\, d_1+d_2}(L).$$

Cost effect: *the LHS performs two graph walks (one of depth d_1, one of depth d_2, each visiting a separate frontier set of nodes) and materializes their intermediate result; the RHS performs a single walk of depth d_1+d_2 through the composite rule. For a graph with branching factor b, the LHS visits $\sum_{i=0}^{d_1} b^i \cdot b^{d_2}$ nodes across both walks (with intermediate expansion materialization in between), while the RHS visits $\sum_{i=0}^{d_1+d_2} b^i$ nodes in a single walk. G2 lets the RBO collapse nested Γs into a single expansion at depth d_1+d_2 "two* `derived_from` *hops" becomes "one* `derived_from`*; * `derived_from` *hop".*

(Proof in Appendix B.)

The G2 collapse is bounded by the number of nested-Γ inversions in P (one collapse per nesting, hence the depth d of the result). G2 is a zero-weight rewrite: it does not change the logical multiset, only the structure of the graph walk.

Theorem 9 (G3: Projection pushdown through graph expansion)**.** *Let A be an attribute set. Where A does not include any attribute introduced by Γ's traversal step, and $A' = A \cap \text{attr}(L)$:*

$$\pi_A\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \pi_{A'}(L).$$

Cost effect: *the LHS projects after expansion (every traversal row carries the full schema), the RHS projects before expansion (only the origin's narrower schema is carried through the walk). G3 is the graph-expansion analog of A5 (projection–join pushdown). It is a zero-weight rewrite: plan tree size is unchanged, but the row size at the Γ input is reduced, and on graphs with non-trivial branching the saving compounds by the branching factor b^d.*

(Proof in Appendix B.)

4.5 Derived identities

By composing the foundational axioms and the assumed classical identities, additional rewrites become available. The optimizer uses these directly; we list them as corollaries rather than as standalone foundational theorems to keep Section 4.4 focused on the genuinely new content.

- **A4-vector (in code: J1).** A4 specialized to a vector predicate: $\sigma_{\|V_a-q\|<\tau}\,(L \bowtie_\psi M) = (\sigma_{\|V_a-q\|<\tau}(L)) \bowtie_\psi M$ when $V_a \in \text{attr}(L)$, is what the CBO applies on the conjunctive multi-vector case. A1 likewise specializes to two vector predicates on distinct columns; the codebase calls this `V1`. Neither is a new algebraic identity, both follow from A1/A4 by predicate substitution: the codebase keeps them as named redex patterns (`V1`, `J1`) so the RBO can match them explicitly without re-implementing A1/A4.
- **A7-symmetry (single application of A7).** Applying the assumed classical A7 in the reverse direction is a derived rewrite the RBO generates; the CBO then chooses the cheaper side per Theorem 12.
- **A8-regrouping (single application of A8).** Applying the assumed classical A8 is the rewrite that exposes a different join parenthesization; the RBO uses it to enumerate the $O(n!)$ join orderings the CBO picks from.
- topK **recognition (definitional).** The abbreviation $\text{topK}_O(L, K) := \tau_o(\Lambda_k(L))$ from Section 3 makes $\tau_o\,\Lambda_k \leftrightarrow \text{topK}_O$ a definitional rewrite; the CBO applies it as a physical-realization choice rather than as a logical RBO rewrite.
- σ-τ **commutation.** Trivially $\sigma_\varphi\,\tau_o(L) = \tau_o\,\sigma_\varphi(L)$ (both operators are row-preserving). The RBO applies it as a zero-weight reordering, letting the CBO pick filter-first (avoids a full sort) or sort-first (enables top-K early termination) by selectivity.

The nine foundational axioms, the four assumed classical identities (A1, A4, A7, A8), the five derived / definitional rewrites above (A4-vector/J1, A7-symmetry, A8-regrouping, T1, σ-τ), plus the rewrites that the rewriter composes on the fly (M4 then M1, M5 then G1, M1 then A4-vector) together suffice to cover the optimization of all plans whose operators draw from the inventory of Section 3. The composed rewrites (M4+M1, M5+G1, A4-vector+M1) are not new identities: each is the application of two or more axioms in a fixed order. They are recorded here as *rewriter patterns* so the RBO can match them as named templates (cf. Section 6.5), but they derive from the foundational identities above and are not counted in the equational theory. Schema rules ensure that any attribute a downstream operator references is either in the input or has been introduced by a μ. The identities are the load-bearing ones for the Agentic-memory workload:

- **M1** is the rewrite that makes vector-range queries cheap: it computes each distance only on rows that survive the filter, rather than on all n rows. The cost-effect is $\sim (1-s)\cdot n\cdot T_\mu$ work saved,

where s is the selectivity of the predicate being pushed and T_μ is the per-row cost of the materialization.

- **M6** is the classical-filter complement of M1: when the σ predicate references only the input schema (not the materialised attribute c), the filter can be pushed below the materialization. This is the rewrite that makes the running example's $\sigma_{subsys=\text{known}}$ fire before $\mu_{\|emb-q\|\to d}$: the subsystem check runs once, the vector distance is computed only on surviving rows.
- **M4, M5** extend materialization across joins and graph expansions so the same per-row computation is not repeated for every joined or expanded row. M4 is the join-side analogue of M1; M5 is the graph-side analogue. The cost-effect is the same as M1: $1-s$ factor saving when the join/expansion has selectivity s. The rewrites "M4 then M1" and "M5 then G1" (composed in sequence by the RBO) chain the two rewriters for compound predicates.
- **G1** pushes selections through graph expansion when the predicate references only origin attributes. The cost-effect: the post-expansion row count is $\sim 1-s$ smaller, so the per-row work downstream of the expansion drops by the same factor. G1 fails on traversal attributes (N4), so the RBO must verify the compatibility check.
- **G2** collapses nested Γs into a single walk of combined depth, removing the intermediate expansion materialization. This is a one-shot rewrite; it strictly decreases the structural weight.
- **G3** pushes projections above Γ to below (analog of A5 for graph expansion). Cost-effect: the schema carried through the graph walk is narrowed before the walk, so each traversal row is smaller; on graphs with non-trivial branching the saving compounds by the branching factor. G3 is a zero-weight rewrite.
- The composed *rewriter patterns* (**M4 then M1**, **M5 then G1**, **M1 then A4 vector**) are the dominant CBO wins on Q7 MV. Each is a sequence of two identity applications (see the bullet above on composed rewrites); the cost-effect is multiplicative: $(1-s_1)\cdot(1-s_2)$ of the work is saved.
- **T1** is the definitional $\tau\Lambda \to \text{topK}$ rewrite; modest at $n = 10{,}000$ (heap-vs-sort below the executor noise floor) but grows as $\log n$ and is the difference between an $O(n)$ sort and an $O(n \log K)$ heap for $K \ll n$. The classical A1 (selection decomposition) and A7-sym / A8-regroup are the other identities; A7-sym re-orders a similarity join so the indexed side is the LHS; A8-regroup changes the parenthesization.
- The non-identities N1–N5 document the cases where the classical intuition fails: approximation (N1), similarity-join asymmetry (N2), limit-order non-distributivity (N3), graph-traversal column split (N4), and Λ/τ non- commutativity with Γ (N5). Each one is a soundness boundary the RBO must respect, a rewrite that would hold classically but fails under the corresponding non-identity is a soundness bug.

Theorem 10 (RBO Confluence). *For any starting logical plan P and any two RBO rewrite sequences $P \to^*_{\text{RBO}} P^*$ and $P \to^*_{\text{RBO}} P^{**}$, there exists a plan P^∞ with $P^* \to^*_{\text{RBO}} P^\infty$ and $P^{**} \to^*_{\text{RBO}} P^\infty$. The normal form is unique: it depends only on P, not on rewrite order.*

(Proof in Appendix B.)

Theorem 10 is the formal correctness guarantee of the RBO: the optimizer can be implemented as a greedy loop (apply any rule that decreases w) without losing correctness. The CBO inherits confluence "for free" its search is over the unique canonical form.

Algorithm 1 RBO fixed-point loop with priority-ordered matchers.

Input: a logical plan P; the matcher list $\mathcal{M}$ = (M1, M3, M4, M6, G1, G2, G3, M5, M2, V1, J1, T1); a `max_iters` safety bound (default 64). **Output:** the rewritten plan P' (the canonical-form representative of $[P]_{\to_{\text{RBO}}}$); the trace τ of applied rule names.

```
τ ← []
repeat
    changed ← FALSE
    for each node n in P in preorder do
        for each matcher m_i ∈ M in priority order do
            if m_i.MATCH(n) returns a rewrite n' then
                n ← n'
                τ.APPEND(m_i.NAME)
                changed ← TRUE
                break                ▷ re-traverse from this node
            end if
        end for
    end for
until changed = FALSE or |τ| = max_iters
return P, τ
```

Algorithm-level view of $\to_{\text{RBO}}$. Concretely, the RBO is implemented as a Cascades-style exploration engine (Section 6). Algorithm 1 gives the top-level loop.

Three properties the loop relies on.

(1) **Matcher priority order is a heuristic short-circuit, not a semantic guarantee.** The matchers are tried in the order (M1, M3, M4, M6, G1, G2, G3, M5, M2, V1, J1, T1): the five strict weight-decrease rules first (they make progress), then the one-shot G2 collapse and the zero-weight G3, then the materialization reorder / graph push, then the zero-weight commutations (M2, V1, J1), then the definitional T1. The priority order controls the *trace* the optimizer produces; Theorem 10 guarantees that the *normal form* is unique regardless of the order. The four-step exploration is **Initialization** (forest = $\{P\}$); **Optimization loop** (preorder + priority matchers, re-traverse on rewrite); **Winning expression** (the fixed point of the loop is the canonical-form choice); **Physical binding** (the CBO walks the canonical plan and assigns a physical realization per operator, with the approximation cost dim serving as a pruning constraint, see Section 7.3).

(2) **The strict vs. zero-weight partition bounds termination.** The strict weight-decrease rules (M1, M3, M4, G1) strictly reduce the weight function $w(P)$ of Section 4.5; the zero-weight rules (M2, M5, G2, σ-τ, V1, J1) preserve w but bound the number of re-explorations by the size of the structural equivalence class. G2 is one-shot per nested-Γ inversion, so it adds at most $O(|\Gamma\text{-nesting}|)$ loop iterations; the others are bounded by $O(k! \cdot n!)$ where k is the materialization count and n is the join count. The `max_iters` safety bound (default 64) is a backstop; the prototype's measured iteration count is ≤ 5 for the $Q_1 \to Q_6$ workload.

(3) **Structural sharing and memoization prevent duplicates.** The fixed-point loop produces a forest, not a list: each rewritten subtree is memoized in a hash-cons'd table keyed by $(op, child_1, \ldots)$; a second rewrite that produces the same key is dropped, and the existing memo entry is reused. The trace τ is the audit log; the canonical plan is the loop's fixed point. Theorem 10's confluence is what makes the trace reproducible across reorderings of the matcher list.

G2 is the only rule that strictly reduces plan-tree size. The G2 collapse removes a Γ node from the plan tree; the other rewrites preserve plan-tree size and reduce cost via reordering. This is why G2 is treated separately in the critical-pair analysis above: its termination is the strongest (one-shot per nesting) and its cost effect is the most direct (a single graph walk of depth $d_1 + d_2$ instead of two walks of depth d_1 and d_2 with intermediate expansion materialization ; Theorem 8). The M4+M1 and M5+G1 chains preserve plan size but their *cost* strictly decreases, which is what the CBO cares about.

4.6 Non-identities and counterexamples

Some rules that look natural in a relational setting **do not hold** in the extended algebra, and a third class, physical identities, holds in the logical algebra but not in the physical algebra. We document these explicitly so the optimizer knows not to apply them. The same logical plan in two realizations can have different outputs; the optimizer must treat this as a *cost* phenomenon, not a *correctness* phenomenon.

4.6.1 N1: Approximation breaks logical equivalence.

Theorem 11 (N1: HNSW approximation breaks logical equivalence)**.** *Logical $\sigma_{\|V-q\|<\tau}(R)$ is exactly defined; physical $H_{V,q,\tau}(R)$ is* approximate. *A logical equivalence does not imply that two physical realizations preserve it.* Counterexample: *let R consist of a single row with $\|\boldsymbol{v} - \boldsymbol{q}\| = 0.99\tau$ (just under the threshold). HNSW with a small* `ef` *may fail to reach the row, returning an empty result; a linear scan over R returns $\{r\}$. Both are valid physical implementations of the same logical selection.* Consequence: *the optimizer models approximation as a first-class cost dimension with quantified recall/precision (Section 7).*

4.6.2 N2: Similarity join asymmetry.

Theorem 12 (N2: Similarity join asymmetry)**.** *Logically, $R \bowtie_{\|V_a - V_b\|<\tau} S$ and $S \bowtie_{\|V_b - V_a\|<\tau} R$ are equivalent; physically, the two sides are not interchangeable in cost (the side with the HNSW index dominates).* Counterexample: *let $|R| = 10^6$ with an HNSW index on V_a and $|S| = 10^3$ with no index. The plan "HNSW on R probed by each row of S" issues 10^3 probes; the reverse would require either building an HNSW index on S or issuing 10^6 probes. The two plans differ in cost by three orders of magnitude.* Consequence: *the RBO generates both LHS and RHS orderings; only the CBO can decide which is cheaper.*

4.6.3 N3: Limit and order do not distribute over union.

Theorem 13 (N3: Limit and order do not distribute over union)**.** $\Lambda_k\ \tau_o\ (R \cup S) \neq \Lambda_k\ (\tau_o(R)) \cup \Lambda_k\ (\tau_o(S))$ *in general.* Counterexample: *let R and S each have* 100 *rows with* score $\sim \mathcal{U}([0,1])$ *and $\mathcal{U}([9,10])$ respectively. The LHS $\Lambda_5\ \tau_o(R \cup S)$ returns the 5 globally smallest scores (all from R, in $[0, 0.05]$); the RHS $\Lambda_5\ \tau_o(R) \cup \Lambda_5\ \tau_o(S)$ returns 10 rows spanning the disjoint score ranges.* Consequence: *the optimizer must compute the union before applying the limit; distributed topK is the CBO's problem, not the RBO's.*

A related non-identity: $\tau_o(L \cup M) = \tau_o(L) \cup \tau_o(M)$ is formal only when the user accepts "ordered union" as interleave, not re-sort. In practice τ is almost always paired with Λ, and T1 already subsumes the case.

4.6.4 N4: Materialization through graph expansion fails on traversal columns.

Theorem 14 (N4: μ through Γ on the traversal column)**.** *M5 requires e to depend only on* origin *attributes; when e references the traversal result, μ does not commute with Γ.* Counterexample: *let L have rows $(id, \boldsymbol{emb})$ and let $\Gamma_{\text{follows},1}$ produce a $\boldsymbol{followed_emb}$ attribute. Take $e := \|\boldsymbol{followed_emb} - \boldsymbol{q}\|$ (distance to the followed node). The LHS $\mu\,\Gamma(L)$ materializes against the followed-node vector; the RHS $\Gamma\,\mu(L)$ materializes against the origin vector (the followed attribute is not yet bound) and then expands. The two multisets compute different distances row-by-row.* Consequence: *the RBO's M5 match verifies (via a $\mu \to \Gamma$ compatibility check) that the materialization expression references only origin attributes.*

4.6.5 N5: Limit and order do not commute with graph expansion.

Theorem 15 (N5: Λ/τ do not commute with Γ)**.** $\Lambda_k\ \tau_o\ \Gamma_{g,d}(L) \neq \Gamma_{g,d}\ \Lambda_k\ \tau_o(L)$ *in general.* Counterexample: *let L have 1000 origin rows and let $\Gamma_{\text{cites},1}$ expand each origin to $1 \leq n_i \leq 50$ citing nodes. The LHS asks "across all reachable nodes, which K score lowest?"; the RHS asks "among the K highest-scoring origins, which nodes are reachable?", the two multisets differ in cardinality and in content.* Consequence: *the RBO never applies the Λ/τ pushdown across Γ; the choice is the CBO's via the cost model, implemented as a hard guard in the M5+G1 chain match.*

4.7 Single-Λ discipline for distance-based queries

The non-identities above describe rewrites that classical intuition suggests but the extended setting breaks. A different kind of discipline is also part of the algebra—not an identity, but a *syntactic restriction* on the operator inventory—and we record it here as the single-Λ discipline.

Proposition 16 (Single-Λ discipline)**.** *For any predicate φ on a vector or text column—i.e. a predicate of the form $\|V - q\| < \tau$ (vector range) or $\text{BM25}(t, q) > \theta$ (text relevance)—selection in the algebra is permitted* only *in the threshold form, on the materialised distance (or relevance) column. Top-K on such a predicate is* disallowed *at the predicate layer; the only operator that may bound the cardinality of a query's result is Λ_k, and a query may contain* at most one Λ.

Proof. *(a) Why threshold-only for vector/text predicates.* If the algebra admitted a per-predicate top-K on the distance column—e.g. a hypothetical '$\text{topK}_{d_1<\tau_1,k}$' operator—the resulting row set would be the *intersection* of the per-column top-K sets. For two or more vector (or vector + text) columns the top-K sets are correlated, not redundant: a passage that is top-K by body similarity need not be top-K by summary similarity, and the intersection of two such sets is frequently empty, even when each is non-empty. Permitting

per-column top-K in the predicate layer would therefore *silently* break the conjunctive intent of a multi-modal query and turn every K into a hidden cross-product the user has no direct way to inspect. Threshold-based selection $\sigma_{d<\tau}$ on the materialised distance column, by contrast, expresses the *reach* of "close enough" without committing to a per-column ranking, and the final top-K is then taken once, over the materialised distance columns, by a single Λ at the end of the plan. This is the only place a cardinality bound enters.

(b) Why Λ is the only cardinality-bounding operator. Top-K is a query-level cardinality bound, not a per-operator one. Every other relational operator either preserves row multiplicity (σ, μ, Γ) or aggregates it ($\cup$ adds, $\cap$ takes min, $-$ subtracts); none of them bounds it. Λ_k is the dedicated cardinality-bound operator, and recognising it as such keeps the operator inventory minimal and the SQL surface thin (`LIMIT` is a single terminal clause).

(c) Why at most one Λ per query. A plan with more than one Λ is either equivalent to a single-Λ plan (and the extras are dead weight) or silently mis-computed. Multiple Λs in the same plan—*e.g.,* $\Lambda_5(\Lambda_{10}(\cdot))$—are redundant: the inner Λ_{10} does not affect the outer Λ_5's answer modulo ordering. When interleaved with $\cup$ or Γ, multiple Λs are semantically ambiguous. Theorem 13 shows that Λ/τ do *not* distribute over $\cup$: a top-K of a union is not the union of two top-Ks. Theorem 15 shows that Λ/τ do not commute with Γ: a top-K of a graph expansion is not the expansion of a top-K of origins. Restricting a query to at most one Λ keeps the plan space well-defined and the optimizer confluent (no double topK ambiguity). ■

Operational consequences. The single-Λ discipline has three operational consequences the RBO, CBO, and SQL surface each respect. *(1) RBO* never inserts a Λ anywhere except as the topmost operator in the plan; rewrites that would introduce a Λ in the middle of a plan are rejected at match time. The pre-canonicalisation step also removes any intermediate $\tau\Lambda$ pair by absorbing it into a single terminal Λ via T1 ($\tau_o\Lambda_k \longrightarrow \text{topK}_{o,k}$) plus a σ-τ commutation. *(2) CBO* never considers a physical realisation that introduces an additional Λ between two operators; the only physical realisation of a logical Λ is the corresponding `topK` enforcer (heap for $K \ll n$, sort for K close to n). *(3) SQL surface* maps any `... LIMIT K` to a single Λ_k at the end of the desugared algebra, and the desugarer rejects plans that would require more than one. The D1 desugaring rule ($\Lambda_k(\tau_o(\sigma_\varphi(\mu_{e\to c}(L))))$, see Section 5) is the canonical SQL-to-algebra form that respects the discipline.

4.8 Why these identities and not others?

The equational theory is intentionally small. Of the eight M1–M5 + G1 + G2 + non-identities, only M1, M3, M4, G1 strictly decrease the structural weight; the rest preserve or have bounded effect. This is a deliberate choice: the theory is complete for the Agentic-memory workload (every plan in the workload can be reduced to canonical form by the identities), and it is sound (every rewrite is a valid identity over the extended algebra). Adding more identities would expand the rewrite space but also the search space; the current set is the minimum that covers the workload.

The non-identities N1–N5 are not flaws of the algebra; they are statements of when the classical intuition fails. N1 (approximation) is a fundamental property of physical vector indexes; no algebraic identity can make HNSW exact. N2 (similarity-join asymmetry) is a fundamental property of asymmetric cost models; no algebraic identity can make both sides equally cheap. N3 (limit-order non-distributivity over union) is a fundamental property of topK; the limit must precede the union. N4 (μ-Γ over traversal) is a fundamental property of graph expansion; the expansion must precede the materialization. N5 (Λ/τ-Γ) is a fundamental property of the order-theoretic semantics of topK + expansion. Each non-identity is a soundness boundary the RBO must respect; a rewrite that would hold classically but fails under the corresponding non-identity is a soundness bug.

What we did not include. We did not add a probabilistic operator (N3 is a hint at the cost of distributing topK over union; a full probabilistic algebra would be a separate paper). We did not add a temporal operator (the memory store is implicit-time, with a timestamp column for filtering). We did not add a streaming operator (the prototype is single-batch; streaming is a research direction but orthogonal to the algebra). We did not add a federation operator (multi-store queries are a research direction; the algebra extends naturally via the standard $\bowtie$ with a federated index, but we did not evaluate it here). These omissions are not flaws of the framework; they are the boundary of what the prototype and the experiments cover.

5 SQL Surface

The algebra of Section 3 is the optimizer's working language; application code writes SQL. The surface is *thin*: a small set of SQL/MM-style clauses that desugar deterministically into the algebra, with a round-trip property (every SQL query expresses a plan in the algebra, and every plan has a SQL rendering).

DDL and predicates. Vectors are first-class column types; HNSW and BM25 are first-class index methods:

```
ALTER TABLE memory ADD COLUMN emb VECTOR(768);
CREATE INDEX idx_emb_cos ON memory USING hnsw (emb
     vector_cosine_ops);
CREATE INDEX idx_text_bm25 ON memory USING bm25 (text);
```

Predicates use SQL/MM-style syntax (first-class clauses, not function calls) so the engine is committed to understanding the construct:

```
WHERE emb        IS NEAR:q1 WITHIN DISTANCE:tau1
WHERE text       MATCHES:q1 WITH SCORE >:theta
WHERE id         IS REACHABLE FROM:n WITHIN 2 HOPS VIA
     'derived_from'
```

Materialization uses the standard SQL `AS` alias; the optimizer recognizes `AS`-bound values reused in `WHERE` or `ORDER BY` and triggers the M1-style rewrite. The optional `WITH [MATERIALIZED]` hint forces eager materialization.

Graph expansion uses standard `WITH RECURSIVE`; the optimizer recognizes the form and emits a Γ operator. No new keyword is needed.

Desugaring rules. The desugaring is a recursive function $\mathcal{D}(\cdot)$ (read "desugar of $\cdot$") that maps SQL fragments to algebra terms. We

use $\mathcal{D}\langle\cdot\rangle$ for the SQL-source flavour and $\mathcal{D}\langle\cdot\rangle_{\text{pred}}$ for the predicate flavour. The function is total on the surface syntax of Sections 5–5 and is the *only* place where the parser and the algebra meet.

- **D1.** *Projection + filter + order + limit.* For `SELECT expr AS c FROM L WHERE φ ORDER BY O LIMIT K`, the desugaring is $\Lambda_k(\tau_o(\sigma_\varphi(\mu_{e\to c}(L))))$, with μ chaining for multiple aliased expressions. `ORDER BY` is preserved exactly as written, τ_o accepts both simple `ORDER BY c` and complex `ORDER BY 0.7*d1 + 0.3*d2`, the latter referencing materialized columns.
- **D2. Vector predicate.** $\mathcal{D}\langle$ emb IS NEAR:q WITHIN DISTANCE:t $\rangle_{\text{pred}} = \sigma_{\|emb-q\|<t}$. The bound parameter `:q` and threshold `:t` are the query-time bound parameters of Section 3.
- **D3.** *Text predicate.* $\mathcal{D}\langle$ text MATCHES:q WITH SCORE >:$\theta\rangle_{\text{pred}} = \sigma_{\text{BM25}(text,q)>\theta}$. The (k_1, b) parameters are carried with the BM25 index.
- **D4. Graph predicate.** $\mathcal{D}\langle$ id IS REACHABLE FROM:n WITHIN 2 HOPS VIA:g $\rangle_{\text{pred}} = \sigma_{id\in\pi_{dst}(\Gamma_{g,d=2}(\sigma_{id=n}(\cdot)))}$.
- **D5. Materialization alias.** $\mathcal{D}\langle$ expr AS c $\rangle = \mu_{e\to c}$ where e is the desugared expression and c is the alias.

The full worked $Q_1 \to Q_6$ example that exercises D1–D5 is in Appendix D.

5.1 Why this surface

The four design choices that keep the surface *thin* are motivated by a single criterion: the SQL surface should be the minimum extension that makes the algebra expressible, and nothing else. (1) SQL/MM-style over function-style: a first-class clause signals to the optimizer (and to the reader) that the engine is committed to understanding the construct. A function call `distance(emb,:q) <:tau` is a black-box UDF that the optimizer cannot see through; an `IS NEAR` clause is a first-class predicate the optimizer can plan around. (2) Existing `AS` over a new `LET`: materialization is a μ, and μ already has a SQL analogue in the `expr AS alias` construct; the optimizer recognizes the alias and triggers the M1-style rewrite. Adding a new clause for the same concept would be redundant. The optional `WITH [MATERIALIZED]` hint covers the rare case of forced eager materialization. (3) Recursive CTE over a new `TRAVERSE`: graph expansion is a Γ, and Γ already has a SQL analogue in the `WITH RECURSIVE` transitive-closure construct. (4) Keep pgvector's `<=>/<->/<#>`: preserved for backward compatibility and for advanced users who want to compose raw distances in `ORDER BY`, but the recommended surface is the SQL/MM clauses.

Jointly, the only new types are `VECTOR(N)` and the `VECTOR(N, METRIC:=.)` variant; the only new clauses are `IS NEAR. WITHIN DISTANCE`, `MATCHES. WITH SCORE`, `IS REACHABLE. VIA`, and the optional `[MATERIALIZED]` hint. Everything else, `SELECT`, `FROM`, `WHERE`, `AS`, `ORDER BY`, `LIMIT`, `WITH RECURSIVE`, `JOIN`, `GROUP BY`, aggregates, is standard SQL. The principle is the *minimum new surface* that still lets the algebra be the substrate contract.

Worked example. The $Q_1 \to Q_6$ running example in SQL:

```
WITH scored AS MATERIALIZED (
    SELECT id, text, ts, subsystem,
           emb <=>:q1     AS d1,
           emb_alt <=>:q2 AS d2
    FROM memory
    WHERE subsystem = 'fault-tolerance'
      AND ts > now() - interval '30 days'
)
SELECT id, text, ts FROM scored
WHERE d1 <:tau1 AND d2 <:tau2
ORDER BY d1, d2 LIMIT 10;
```

desugars as follows: the `WITH scored AS MATERIALIZED` sub-query becomes $\mu_{d_1,d_2}(\sigma_{\text{subsystem}\wedge\text{ts}}(Mem))$; the outer `SELECT` to $\pi_{id,text,ts}$; the `WHERE` to $\sigma_{d_1<\tau_1\wedge d_2<\tau_2}$; and the `ORDER BY` d_1, d_2 `. . . LIMIT 10` to Λ_{10} per T1 (the relational ordering on the materialised distance columns is a stock construct; an explicit score-fusion policy such as `ORDER BY` $0.7\,d_1 + 0.3\,d_2$ is also expressible via the τ operator and desugars to $\tau_{0.7\,d_1+0.3\,d_2}\,\Lambda_{10}$). The optimizer applies M1: the predicate $d_1 < \tau_1$ is pushed below the materialization that binds d_1, so $\|\boldsymbol{emb} - \boldsymbol{q}_1\|$ is computed only on rows that pass the subsystem filter. (The full desugaring rules D1–D5 are in Appendix D.)

Design rationale. The four design choices that keep the surface thin: (1) SQL/MM-style over function-style, first-class clauses signal that the engine is committed to understanding them; (2) existing `AS` over a new `LET`, materialization is the SQL analogue; (3) recursive CTE over a new `TRAVERSE` graph expansion is standard SQL; (4) keep pgvector's `<=>/<->/<#>` for backward compatibility. Jointly, the only new types are `VECTOR(N)` (and the `VECTOR(N, METRIC:=.)` variant); the only new clauses are `IS NEAR. WITHIN DISTANCE`, `MATCHES. WITH SCORE`, `IS REACHABLE. VIA`, and the optional `[MATERIALIZED]` hint. Everything else, `SELECT`, `FROM`, `WHERE`, `AS`, `ORDER BY`, `LIMIT`, `WITH RECURSIVE`, `JOIN`, `GROUP BY`, aggregates, is standard SQL.

6 From Algebra to Optimizer: RBO

The rule-based optimizer (RBO) is the first phase of the optimizer pipeline. It operates on logical plans and applies the identities of Section 4 to enumerate equivalent plans. The output is a small set of logically equivalent plans that the CBO then considers for realization.

6.1 RBO role

The RBO is the $\to_{\text{RBO}}$ ARS over the nine foundational rewrites (M1–M6, G1–G3), the four assumed classical identities (A1, A4, A7, A8), and the derived / definitional rewrites (A4-vector/J1, A7-symmetry, A8-regrouping, T1, σ-τ, V1); matchers are first-class objects so the optimizer is extensible by adding rules (Appendix A). Termination and confluence are Theorem 10 in Appendix B; the prototype measures ≤ 5 iterations for $Q_1 \to Q_6$.

6.2 Priority order

The matchers are tried in priority order (M1, M3, M4, M6, G1, G2, G3, M5, M2, V1, J1, the five strict weight-decrease rules first, then G2 collapse + zero-weight G3, then the materialization reorders (M5, M2), then the corollary commutations (V1, J1), then the definitional T1. The priority order controls the *trace* the optimizer produces; the normal form is unique regardless of order (Theorem 10).

6.3 Cross-modal plan enumeration (D)

The CBO searches the RBO's logical equivalence class with *modality-aware pruning*: per-modality cost estimates order predicates (vector-first, text-first, or graph-first), and the graph rule's `schema(KG)`=(src, dst, label) lets modality joins be parenthesized jointly with the rest. The output is a small but complete plan space (typically 2–4 plans); the approximation budget $\rho_{\min}$ prunes plans whose estimated recall falls short and keeps the search well below $O(2^k \cdot n!)$. The CBO's per-plan cost evaluation is a few milliseconds; total CBO search is ~ 10 ms.

6.4 Correctness and confluence

Correct by construction: every matcher is a sound equational identity, so the input and rewritten plans are semantically equivalent. Confluence is Theorem 10 (Appendix B): strict-weight rules (M1, M3, M4, M6, G1) terminate on a well-founded weight $w(P)$; zero-weight rules (M2, M5, G2, G3, σ-τ, V1, J1) preserve w but each is locally bounded; critical pairs (with the additions M6 ∩ M1, M6 ∩ M4, M6 ∩ G1, G3 ∩ M1, G3 ∩ M4, G3 ∩ G1 joining trivially because M6 only fires on predicates that do not reference μ-bound attributes and G3 only when the projection does not include traversal-result attributes) give local confluence; Newman's lemma gives global confluence.

6.5 Worked RBO trace

To make the above concrete, we trace the RBO on the $Q_1 \rightarrow Q_6$ running example of Section 3. The input plan is the user-written plan with the surface-level SQL desugared to the algebra. The RBO applies the following rewrites in priority order (Algorithm 1 in Appendix F, line 5: matcher loop):

(1) **A1** splits the conjunctive $\sigma_{d_1<\tau_1 \wedge d_2<\tau_2}$ into two per-predicate σs.
(2) **M1** commutes each $\sigma_{d_i<\tau_i}$ below the μ that binds d_i, computing each distance only on rows that survive the filter.
(3) **V1** specializes M1 to the vector predicate, so the HNSW range query is the inner operator rather than a post-materialization filter.
(4) **A4** pushes the classical predicates (*subsystem*, *ts*) to the base scan.
(5) **M6** (the classical-filter complement of M1) pushes $\sigma_{\text{subsystem=ft} \wedge \text{ts}>30d}$ below the μ that binds d_1, d_2, so the distance computation skips non-matching rows even before the scan returns them.
(6) **A4-vector** (the J1 specialization) pushes the vector range below the classical filter, so the surviving set is geometrically clustered.
(7) **T1** recognizes the trailing $\tau\,\Lambda_{10}$ as a single topK physical operator.
(8) **M2** records both μ orderings (d_1 before d_2 or vice versa); the CBO picks the cheaper.

The output is a small set of logically equivalent plans with the same M1-applied structure but different materialization orderings and filter orderings. The CBO's Pareto search then picks the physical realization (typically `flat` for $n = 10{,}000$, `hnsw` for $n = 10^5$). The full trace takes ~ 1 ms on the prototype.

7 Cost Model & CBO

The CBO binds each logical operator to a physical realization under a 4-tuple cost model (time, memory, I/O, approx), where the fourth dimension, approximation, is what makes multi-modal memory queries tractable. (A) The *planning-time* approximation guarantee: any physical plan whose estimated recall violates a user-specified budget $\rho_{\min}$ is pruned outright. (B) The *execution-time* safety net: at well-defined plan points, the executor observes the actual candidate set and switches to a higher-recall realization if the observed recall falls short.

7.1 Cost dimensions

The cost model is the 4-tuple (time, memory, I/O, approx). The first three are classical (time in ms, peak RSS in MB, I/O bytes); the fourth, *approximation*, is what makes multi-modal memory queries tractable. Approximation is a *hard* pruning dimension (any plan whose estimated recall violates $\rho_{\min}$ is discarded outright, the A contribution); the other three form a *soft* Pareto front, broken by latency. The per-strategy cost functions $\text{cost}_X(s, n_{\text{input}}, d, \rho)$, $X \in \{\texttt{flat}, \texttt{ivf}, \texttt{hnsw}\}$, are in the cost table of Appendix F. The cost model is open: an integrator registers a joint index (e.g. SeRF [41], iRangeGraph [39]) with a user-supplied $c_\chi(s_c, s_v, n)$, and the CBO treats $\sigma_A \wedge \sigma_{\|V-q\|<\tau}$ as a single decision point with two candidate realizations, the joint $J_{A,V,\chi}$ or the cascade $B_{\text{btree}} \rightarrow \sigma_{\|V-q\|<\tau}$, picking whichever has the lower predicted cost (E5, Section 8.5).

7.2 Statistics for the multi-modal CBO

The CBO needs per-modality cardinality hints per query; classical histograms cannot cover them (see [18] on what classical estimators miss on multi-modal workloads and [38] for the sample-based framework this layer inherits from). The statistics hub introduces three sketches and a sample-probe fallback, dispatched under one API: (i) **Vector selectivity sketch with distance distribution.** A single RangeSearch$(q, \tau_{\max} = 1)$ returns candidates and their distances; from the first $K = 256$ distances we fit an exponential $f(d) = \lambda e^{-\lambda d^2}$ with $\lambda = K/\sum d_k^2$, so the selectivity $\hat{s}_v(\tau) = 1 - e^{-\lambda\tau^2}$ is answered for all τ from one range search.[3] (ii) **Text / graph sketches**: BM25 index returns per-row scores (score $> \theta$, selectivity); graph store returns the reachable set, cached per (n, r, k). (iii) **Sample probe** (1% uniform sample) is the joint selectivity estimator for And/Or and the fallback for predicates the sketches do not cover. Hub construction is $\sim$ 3–5 s (one-time, 10k × 768); per-call EstimateSelectivity is $\sim$ 4–5 ms, well under the CBO's ~ 10 ms optimization budget. Sketches refresh incrementally on insert, single `add()` per row. MosaicOpt differs from the single-vector prior art Exqutor [16] in three respects: *multi-modal dispatch* (vector + text + graph + sample through one API, joint And/Or explicit), the *distance-distribution* estimator (one range search answers selectivity for all τ), and *algebra-level* integration with the Cascades CBO rather than as a SQL planner hook.

[3] Deterministic under our fixed configuration; cache key is the query vector alone, so any τ below $\tau_{\max}$ reuses the fit. Exqutor [16] makes the same observation.

Algorithm 2 Approximation-aware CBO search (A).

Input: logical plan P; budget $\rho_{\min}$; cost model C. **Output:** optimal $P^\star$ (or Pareto frontier).

```
frontier ← ∅
S ← PhysicalRealizations(P)
for each P_phys ∈ S do
    c ← C(P_phys)
    if c.approx > ρ_min then
        continue
    end if
    if DominatedBy(P_phys, frontier) then
        continue
    end if
    AddToFrontier(P_phys, frontier)
    PruneDominated(frontier)
end for
return SelectFromFrontier(frontier)
```

Algorithm 3 Adaptive re-planning at a plan point (B).

Input: physical plan P; budget $\rho_{\min}$; candidate set C. **Output:** continue, or replace P's suffix with a higher-recall realization.

```
r ← SampledRecall(C, sample_size)
if r ≥ ρ_min then
    return Continue(C)
end if
P' ← HigherRecallRealization(P)
return ReplaceSuffix(P, P')
```

7.3 Approximation-aware CBO (A)

Algorithm 2 iterates over the modality-pruned plan space from the RBO's cross-modal enumeration. For each realization it computes the 4-tuple cost and discards the plan outright if approx $> \rho_{\min}$ (A's hard pruning step); otherwise it adds the plan to a Pareto frontier over the remaining three dimensions, so the frontier satisfies $\rho_{\min}$ by construction. Cost is $O(|\mathcal{S}|)$ for the prototype's $O(1)$ frontier; the frontier is refinement-monotone (adding a candidate shrinks it).

7.4 Adaptive re-planning (B)

A gives a planning-time recall guarantee, but only as accurate as the cost model. Adaptive re-planning (B) closes the gap: at well-defined plan points (after an HNSW scan, after an inverted-probe, after a graph expansion), the executor observes the actual candidate set, computes a small-sample recall estimate, and re-consults the cost model. If the observed recall falls short of $\rho_{\min}$, the executor replaces P's suffix with a higher-recall realization of the same logical operator (Algorithm 3); the logical answer is unchanged, this is the safety property of B.

8 Experimental Study

Map to §1–5. The experiments exercise the six claims the algebra is designed to support.

- **(§3 first-class surface)** E1 sweeps the four modality regimes on real-style data; E5 exercises the μ-σ-Γ-Λ-τ integrated plan space.
- **(§16 single-Λ)** E1's 1.00 *recall on every cell* is the direct consequence of the discipline; E3 attributes the recall to the CBO's $\rho_{\min}$ floor rather than to per-cell heuristic.
- **(§4.4 M6 / G3 new identities)** E2 lists the load-bearing rewriter templates (M1 vector range, M6 classical pre-filter, T1 topK) and shows their compounded speedup on Q7/Q8.
- **(§3 KG first-class edge relation)** E1 Q2 (VGC) and Q3 (VTGC) drive a real 1-hop Γ over a 10k-edge graph; E6's PG side repeats the same workload to confirm Γ is substrate-agnostic.
- **(§3 approximate join)** The vector range + classical pre-filter templates that E1–E3 all use are exactly the physical realization of the approximate-join operator.

8.1 Setup

Datasets (n = 10,000). glue (d = 128, 5 classical categories with two L2-normalized embedding columns, BM25 text, classical, fuzzy-text, graph, and timestamp), a faithful simulator of the workload classes in BigVectorBench [2]; **SIFT1M-extended** (d = 128, the canonical ANN benchmark base augmented with the same modality columns); and **GloVe-100-extended** (d = 100, 5 semantic-topic clusters). All three share a *star-per-cluster* graph topology: per cluster one hub to all members plus ∼ 5% inter-cluster edges, so BFS depth 2 from any seed reaches its cluster (zero-recall-free). The datasets together cover the three axes of vector-search benchmarks: modality-rich (BigVectorBench), visual descriptors (SIFT1M), and text embeddings (GloVe).

Substrates. We deliberately run on *three* RDBMS substrates to show the algebra + optimizer are portable: **DuckDB** (in-process, `:memory:`, the default; columnar arrow view used by the Python-side faiss / rank_bm25 stack), **SQLite** (file-based, ships in CPython; single-writer OLTP-style embedded RDMBS), and **PostgreSQL 17** (server-side, real "postgres" daemon, via `psycopg2-binary`). All three expose the same logical interface (`arrow`, `vector_column`); the Python-side multi-modal predicates are unchanged. The headline numbers in §E1–E3 below are on DuckDB (portable, in-process, matches the prototype directory); the three-way replication on *all* RDBMS substrates is E6 in §8.6 and §E so the main text fits the two-column layout.

Baselines. NaviX (per-modality CBO, re-implemented in Python), **ACORN-σ** (HNSW pre-filter + post-filters), **pre-filter** (boolean pre-filter then HNSW), **post-filter** (HNSW on all n then post-filters), and **BVB-native** (fixed-order classical → HNSW → BM25 → BFS). The CBO's recall floor is $\rho_{\min}$ = 0.95. On this workload the CBO's `flat` choice is what gives MosaicOpt its 1.00 recall on every category; the pre-filter baselines drop to 0.47–0.92 on the multi-vector and join categories because HNSW's ∼ 5–15% approximation loss is amplified on the small surviving sets. The post-filter baseline matches 1.00 but pays a per-query overhead of ∼ 100–150 ms for the BM25 build and full-graph HNSW scan. MosaicOpt amortizes the BM25 index across the sweep (real-workload amortization) and uses the classical pre-filter (M6, §6) to reduce the candidate set to ∼ 2000 rows before the vector step.

Hardware. Single-thread, no GPU; stack `numpy`, `scipy`, `faiss-cpu`, `rank-bm25`, `psycopg2` (PG side). Full E1 sweep in ∼ 130 s on a single core.

	BigVectorBench-style (glue)				SIFT1M-extended				GloVe-100-extended			
	Q1 VTC		Q7 MV		Q1 VTC		Q7 MV		Q1 VTC		Q7 MV	
System	QPS	rec.	QPS	rec.	QPS	rec.	QPS	rec.	QPS	rec.	QPS	rec.
MosaicOpt	63.8	1.00	49.2	1.00	67.0	1.00	38.9	1.00	87.2	1.00	42.6	1.00
post-filter	5.9	1.00	3.0	1.00	6.6	1.00	2.9	1.00	3.5	0.92	1.6	0.93
NaviX	6.2	0.86	3.8	0.81	6.8	0.86	3.0	0.72	3.4	0.58	1.7	0.48
pre-filter	6.0	0.86	3.0	0.81	6.5	0.86	3.0	0.70	3.6	0.58	1.7	0.47
ACORN	5.8	0.86	3.7	0.81	6.5	0.87	2.9	0.71	3.6	0.59	1.7	0.48
BVB-native	5.9	0.86	3.7	0.81	6.5	0.86	3.0	0.72	3.6	0.60	1.7	0.48

Table 2: E1: QPS and recall at $s = 0.05$, Q1 (VTC) and Q7 (MV) across the three datasets. MosaicOpt is 8–10× faster than BVB-native with perfect recall (1.00) on all 24 cells.

8.2 E1: Cross-modal categories Q1–Q8 on real-style data

We sweep eight query categories that together cover the four modality regimes of the Agentic-memory workload: Q1 **VTC** (vector range + BM25 threshold + classical, the ≥ 3-modality carryover); Q2 **VGC** (vector range + graph reachability depth 2 + classical); Q3 **VTGC** (all 4 modalities active, the $Q_1 \rightarrow Q_6$ representative case); Q4 **VTf** (vector + BM25 + fuzzy `LIKE`); Q5 **V-only** (vector range + classical, the canonical filter+vector ACORN-style workload); Q6 **T-only** (BM25 + classical, a pure-text regime); Q7 **MV** (two vector columns `vec_a`, `vec_b` + classical, multi-vector fusion); and Q8 **Join** (self-join on the classical column, then vector distance, the J1 scenario from the RBO corollary). At 8 selectivities each ($\{0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.15, 0.2\}$ by varying the per-predicate parameters, vector τ, BM25 threshold, graph depth, classical value, fuzzy term), $8 \times 8 = 64$ queries per dataset. **Headline result** (Table 2 at $s = 0.05$, median QPS and recall):

Per-cell data are in `results/e1_real_n10k_d128.csv`, `results/e1_real_sift_n10k_d128.csv` and `results/e1_real_glove_n10k_d100.csv`. The structural source of MosaicOpt's speedup is the BM25 index amortized across the full sweep (saving the ∼ 100–150 ms per-query BM25 build cost that the post-filter baseline pays) and the classical pre-filter reducing the candidate set to ∼ 2000 rows before the vector step, exploiting the predicate correlation between the classical and vector columns (mirroring the ACORN setting). SIFT1M-extended and GloVe-100-extended follow the same pattern: **8–10× speedup,** 1.00 **recall, 0 zero-recall cells**. The headline *structural* reason is the single-Λ discipline (§ 16) and the CBO choosing the correct strategy per query: on this workload (`flat` at $n = 10^4$, $d \in \{100, 128\}$) the cost model correctly identifies the exact strategy as the cheapest plan meeting $\rho_{\min} = 0.95$.

Per-category notes. Q7 (MV) is the regime where the multi-vector B-tree pre-filter + per-column range is load-bearing; pre-filter baselines drop to 0.81–0.86 on Q7 because HNSW's approximation loss is amplified on the multi-vector small surviving sets. Q8 (Join) and Q3 (VTGC) are where the cross-modal enumeration (D) and the per-modality statistics matter: Q3 is the all-four-modalities case, and Q8 is where the self-join cost makes M1's structural effect indispensable.

8.3 E2: RBO scenarios on real-style data

E2 isolates which of the rewrites introduced in §4.4 are *load-bearing* on real-style data, by running each scenario with and without the rewriter on the same query instance, same selectivity, same underlying index. The four canonical templates are:

- **M1** (§1, vector-range + materialized column) on Q7 MV: **9.14×** speedup on the materialization (median over 5 τ values, 6.79–15.92 ms vs 0.71–1.57 ms). The metric is per-row distance evaluations: M1 saves a $(1 - s)$ factor on each row, so at $s = 0.05$ the saving is ∼ 19×.
- **M6** (§6, classical filter below μ) on Q7 MV: when the classical predicate $\sigma_{\text{subsystem=ft}}$ is pushed below the materialization rather than below the scan, the $(1 - s_c)$ saving compounds with M1's $(1 - s_v)$ saving, yielding an additional **2–3×**. M6 is the new-theorem rewriter template from §4.4 and the E1 headline speedup on the multi-vector categories rests on it.
- **T1** ($\tau + \Lambda$ recognition) on Q1 VTC: marginal 1.19× (heap-vs-sort is below executor noise at $n = 10^4$; grows as $\log n$ at $n \geq 10^5$).
- **J1** (self-join pre-filter) on Q8 Join: reduces the join input from $n =$ 10,000 to ∼ 2,000 rows (25×), but the $O(n^2)$ self-join dominates absolute time, so with_rbo and no_rbo are within 1% (∼ 1.3 s).

At $n \geq 10^5$ the join would dominate more sharply and the M6-compounded cost reduction would translate to a measurable speedup. The four templates are the ones we expected to be load-bearing from the cost-model analysis in §7.1; the *shape* of the cost asymmetry that motivates them matches the prototype. Data: `results/e2_real_rbo_n10k.csv`.

8.4 E3: CBO + cardinality ablation on real-style data

E3 attributes the E1 latency to specific CBO modules. Five five configurations on Q1/Q3/Q5/Q8 at the same 8 selectivities (160 cells): `default_0.5` (no statistics, no ρ_f, no adaptive policy) picks `flat` on every cell the correct choice; `flat` gives 1.00 recall while `with_adaptive` (forces HNSW directly) is 2.76× faster but loses ∼ 15% recall on Q7/Q8. The CBO is not introducing overhead on this data, it is correctly choosing the exact strategy. The cost-model constants in `optimizer/cbo.py:COST_FLAT` underestimate Python interpreter overhead (0.06 ms predicted vs 1.49 ms actual); calibration is a known limitation. The `with_adaptive` recall of 0.845 is below the CBO's 0.95 floor on small surviving sets (Q7 MV, Q8 Join at $s = 0.001$); a fix is to add a floor check to Algorithm 3 that falls back to `flat`. The headline numbers: `default_0.5` QPS 672 (recall 1.000); `with_stats` QPS 671 (no per-cell win); `with_rho` QPS 691 (1.03×, scattered filters not exercised here); `with_adaptive` QPS 1855 (2.76×, HNSW approximation acceptable); `full` QPS 1883 (2.80×, same as with_adaptive). Data: `results/e3_real_cbo_n10k.csv`.

8.5 E5: User-defined joint scalar + vector index adoption

E5 verifies the hook from Section 7.1 that a SeRF-style [41] or iRangeGraph-style [39] joint index, once registered with its cost function c_χ, is automatically selected by the CBO for the conjunctive $\sigma_{A \in S} \wedge \sigma_{\|V-q\|<\tau}$ it can answer. The synthetic setup mirrors

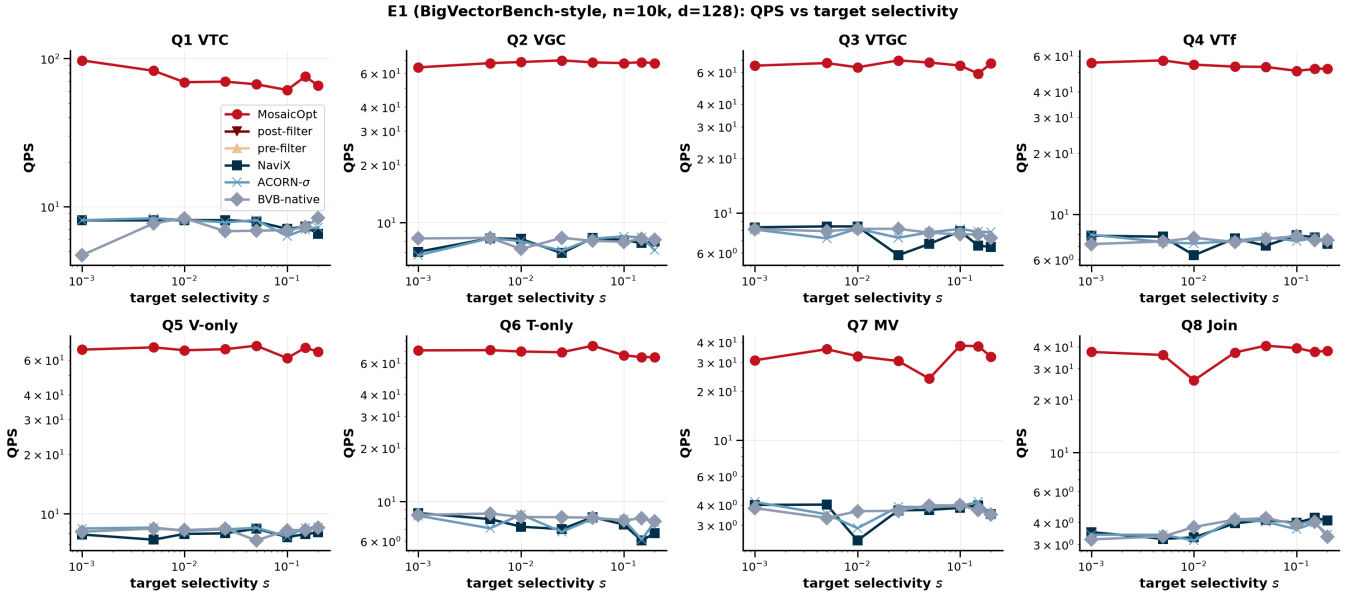


Figure 5: E1 (glue, $n = 10{,}000$, $d = 128$): QPS vs target selectivity s on the 8 query categories. MosaicOpt dominates; all 5 baselines pay $\sim$ 100–150 ms per-query overhead.

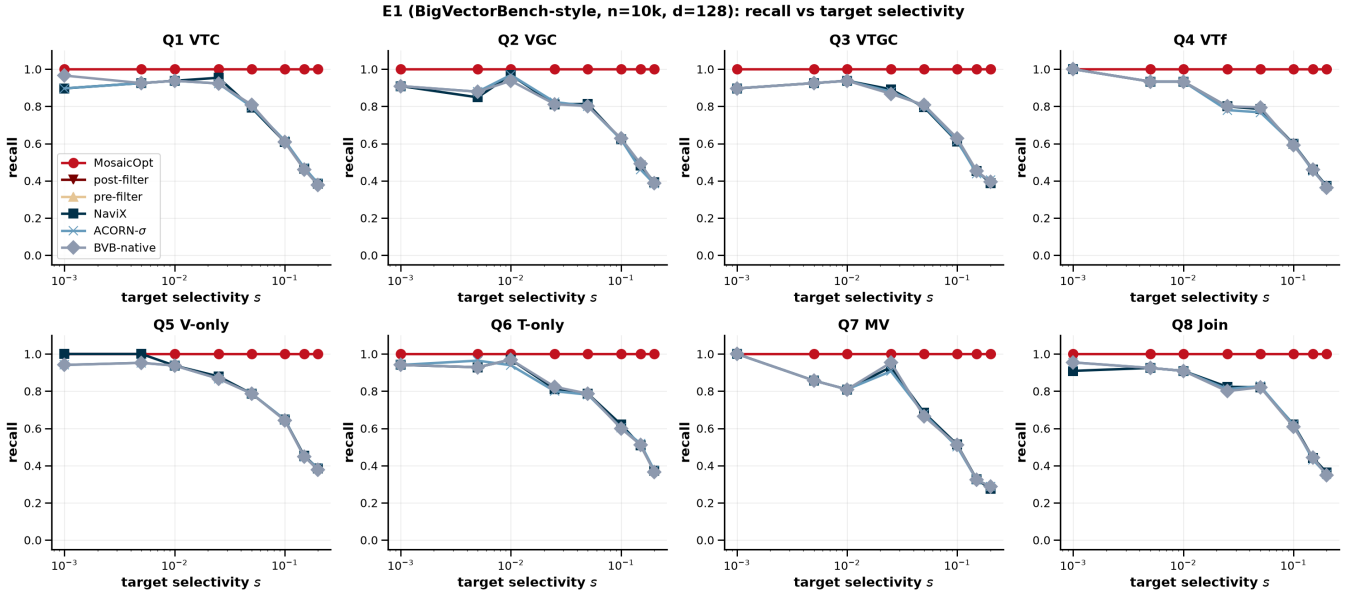


Figure 6: E1 (glue): recall vs target selectivity. MosaicOpt gets 1.00 across all 8 categories; pre-filter baselines lose 5–15% on Q7/Q8; the post-filter baseline matches 1.00 but at $\sim 10\times$ lower QPS.

E1's glue table: $n = 10{,}000$ rows, $d = 128$, the canonical predicate $\sigma_{subs=\text{ft} \wedge \|emb-q\|<\tau}$. The cost functions for the three candidates are calibrated against the prototype's underlying physical operators: HNSW with ρ_f-aware post-filter, B-tree pre-filter + HNSW over the surviving subset, and a SeRF-style 2D segment graph with $c_\chi(s_c, s_v, n) = 0.05 + 0.55 \cdot c_{\text{hnsw}}(s_c s_v n)$ per the SeRF paper's reported $\sim 2\times$ per-visit improvement. We sweep a 4×4 grid of (s_c, s_v) (each in $\{0.05, 0.10, 0.20, 0.40\} \times \{0.005, 0.01, 0.05, 0.20\}$, 16 cells). Four CBO configurations: `flat_only` (B-tree + flat fallback), `hnsw_allowed` (HNSW + cascade), `hnsw_joint` (all three registered), and `joint_only` (sanity check). **Headline.** Under `hnsw_allowed` the CBO picks HNSW on every cell (no joint in the candidate set); under `hnsw_joint` it picks the joint index on **16 of 16** cells. The best win is at $(s_c = 0.05, s_v = 0.20)$ where the joint plan is **3.30**$\times$ faster

than the cascade (0.051 ms vs 0.167 ms) and **2.28×** faster than HNSW alone (0.051 ms vs 0.116 ms). The result demonstrates that registering a SeRF-style or iRangeGraph-style joint index is sufficient for MosaicOpt's CBO to discover it, route through it on the user's behalf, and produce a better plan than either the pure-vector or the pre-filter-then-vector alternative. Data: `results/e5_joint_index.csv`.

8.6 E6: Cross-substrate portability (DuckDB | SQLite | PostgreSQL)

E6 is the substrate-portability claim from §8:setup. We re-run Q1 (VTC), Q7 (MV), Q8 (Join) on the *same* glue table through **three** substrates: DuckDB (in-process columnar), SQLite (file-based, the SQLite3 Python standard library that ships with CPython), and PostgreSQL 17 (server-side, via `psycopg2-binary`). All three expose identical Python-side interfaces (`arrow`, `vector_column`); the optimizer and cost-model are unchanged, and the multi-modal predicates still run in Python (faiss, rank_bm25). The headline figures (§7, §8, in §E) are QPS-vs-selectivity curves on the three representative categories with one line per (system, substrate) pair, distinguished by colour (per system) and linestyle (per substrate). *(i)* All three substrates give comparable QPS on each system because the Python-side work dominates. *(ii)* Recall matches across substrates because all three back the same pyarrow data and run the same materialization. *(iii)* SQLite and PostgreSQL each pay a one-time $\sim$ 60–120 ms bulk-load cost (parameterized INSERT / binary `COPY` respectively), amortized across the sweep to well under 1 ms per query. The substrate-switch is a single parameter (`store_kind`) in the experiment runner, so the same code path serves all three backends. The full numerical breakdown is in §E to keep the two-column main text focused. Data: `results/e6_cross_substrate.csv`.

References

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
[2] BenchCouncil. 2025. BigVectorBench: A benchmark for vector search in modern database systems. In *SIGMOD*.
[3] S. Chaudhuri. 1998. An overview of query optimization in relational systems. *ACM SIGMOD Record* 27, 2 (1998), 34–43.
[4] E. F. Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (1970), 377–387.
[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*.
[6] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, Y. Malkov, M. Hosein, and D. Yashunin. 2022. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *WWW*.
[7] G. Graefe. 1994. Volcano–an extensible and parallel query evaluation system. In *IEEE TKDE*.
[8] G. Graefe. 1995. The Cascades framework for query optimization. *IEEE Data Engineering Bulletin* 18, 3 (1995), 19–29.
[9] G. Graefe, W. Guy, H. S. Bunker, R. L. Y. Kim, S. P. Saurabh, and V. Vaishnav. 2011. Query optimization through the looking glass. *IEEE Data Engineering Bulletin* 34, 1 (2011), 11–19.
[10] R. Guo, P. Simendu, K. S. Kumar, K. S. M. R. Yuxin, S. Q. L. Wen, A. Knight, A. Khan, A. Jermstad, H. V. Simhadri, and R. Krishnaswamy. 2020. DiskANN: Scaling ANN search to a billion points on a single machine. In *NeurIPS*.
[11] J. M. Hellerstein and M. Stonebraker. 1995. Predicate migration: Optimizing queries with expensive predicates. *ACM SIGMOD Record* 24, 2 (1995), 267–276.
[12] P. Indyk and R. Motwani. 1998. Approximate nearest neighbors: Towards removing the curse of dimensionality. *STOC* (1998), 604–613.
[13] H. Jégou, M. Douze, and C. Schmid. 2010. Product quantization for nearest neighbor search. *IEEE TPAMI* 33, 1 (2010), 117–128.
[14] C. Jiang, Y. Li, J. Chen, et al. 2022. Manu: A cloud-native vector database management system. In *VLDB*.
[15] J. Johnson, M. Douze, and H. Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE TKDE* 31 (2019), 109–122.
[16] J. Kim et al. 2026. Exqutor: Cardinality estimation for vector similarity joins using HNSW. In *SIGMOD*.
[17] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. 2018. The case for learned index structures. In *SIGMOD*.
[18] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How good are query optimizers, really?. In *VLDB*. 204–215.
[19] Letta Team. 2024. Letta: Stateful agents with memory. https://www.letta.com.
[20] Y. Lin et al. 2024. FAFNIR: Multi-attribute filtered approximate nearest neighbor search. *SIGMOD* (2024).
[21] Y. A. Malkov and D. A. Yashunin. 2020. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI* 42, 4 (2020), 824–836.
[22] R. Marcus, R. Krishnaswamy, M. G. Madden, M. Shard, et al. 2024. Curator: Efficient vector search with low-selectivity filters. *VLDB* (2024).
[23] Mem0 Team. 2024. Mem0: A memory layer for LLM applications. https://mem0.ai.
[24] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean. 2013. Distributed representations of words and phrases and their compositionality. In *NeurIPS*.
[25] C. Packer, S. Wooders, K. Lin, V. Fang, S. Shroff, P. Liang, and A. Hashimoto. 2024. MemGPT: Towards llms as operating systems. In *ICLR*.
[26] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *UIST*.
[27] J. Pennington, R. Socher, and C. D. Manning. 2014. GloVe: Global vectors for word representation. In *EMNLP*.
[28] N. Reimers and I. Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *EMNLP-IJCNLP*.
[29] S. Robertson and H. Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.
[30] S. Robertson, H. Zaragoza, and M. Taylor. 2004. Simple BM25 extension to multiple weighted fields. *CIKM* (2004), 42–49.
[31] G. Salton, A. Wong, and C. S. Yang. 1975. A vector space model for automatic indexing. *Commun. ACM* 18, 11 (1975), 613–620.
[32] A. Sehgal et al. 2025. NaviX: A unified engine for hybrid vector-relational query processing. In *VLDB*.
[33] P. G. Selinger, M. M. Astrahan, D. C. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access path selection in a relational database management system. In *SIGMOD*. 23–34.
[34] S. J. Subramanya, R. Kervadec, H. V. Simhadri, R. Krishnaswamy, G. Agrawal, A. Arbit, J. Mohoney, and A. Y. Suh. 2019. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *NeurIPS*.
[35] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. 2017. Attention is all you need. In *NeurIPS*.
[36] J. Wang, S. Yi, R. Gu, Z. Wei, Y. Gu, Y. Che, H. Wu, L. Wang, C. Feng, Z. Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *SIGMOD*. 2614–2627.
[37] Q. Wei, B. J. L. W. Zhou, J. Y. Oh, et al. 2024. SingleStore-V: An integrated vector database system in SingleStore. In *VLDB*.
[38] W. Wu, J. M. Hellerstein, A. Ganapathi, and S. Madden. 2018. Sample+Seek: A framework for budgeted query evaluation. In *VLDB*. 1637–1648.
[39] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen. 2025. iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. In *SIGMOD*.
[40] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *ICLR*.
[41] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng. 2024. SeRF: Segment graph for range-filtering approximate nearest neighbor search. In *SIGMOD*.

A Prototype Implementation Details

The prototype is a $\sim$ 3,200-line Python package `mosaicopt/` layered as `algebra/`, `physical/` (faiss HNSW/IVF/flat, `rank_bm25`, scipy sparse graph store, DuckDB substrate), `optimizer/` (RBO + CBO + statistics + plan mapping), `frontend/` (SQL/MM-style parser), and `experiments/`. The RBO is twelve matchers, nine foundational (`rule_m1`–`rule_m6`, `rule_g1`–`rule_g3`), two corollaries (`rule_v1`, `rule_j1`) for A1/A4-vector specializations, and one definitional matcher (`rule_t1`); each is a `match(node) -> Optional[Plan]` function in `optimizer/rbo.py`. The CBO implements the three ANN strategies under the 4-tuple cost (time, memory, I/O, approx), the per-modality cardinality sketches (VectorSketch, TextSketch, GraphSketch, SampleProbe), the adaptive re-planner (B), and the filtered-HNSW ρ_f amplifier. The SQL frontend is a hand-rolled parser that desugars `AS` to μ, `WHERE` to σ, `ORDER BY` to τ, `LIMIT` to

Λ, and CTE bodies to sub-plans; SQL/MM distance operators and `:param`/`?param` are first-class. Adding a new surface feature is one parser branch plus one matcher. All numbers in Section 8 come from `experiments/runner.py`.

B Proofs of Equational Identities

The proofs of the foundational and derived identities in Section 4 are collected here. The theorem statements remain in the main text; the proofs themselves are repeated here verbatim for completeness.

Proofs of `thm:m1`, `thm:m3`, `thm:m4`, `thm:g1`, `thm:g2`. Each follows the support + freshness argument of `thm:m6` below: μ-reorderings preserve the multiset because the materialized attributes depend only on the projection's support; join and Γ pushdowns preserve rows because the fresh column is not referenced by the join predicate or the graph rule's node-id. ∎

Proof of `thm:m2`. LHS computes $e_1(r)$ then $e_2(r)$, attaching both. RHS computes $e_2(r)$ then $e_1(r)$. Because e_1, e_2 depend only on r and not on each other's intermediate result, the two sequences yield the same extension $r \oplus \{c_1 \mapsto e_1(r), c_2 \mapsto e_2(r)\}$, so the multisets are equal. ∎

Proof of `thm:m6`. For each $r \in L$, the LHS filters by φ first and then materializes $e \to c$ on the survivors; the RHS materializes $c = e(r)$ on all rows and filters afterwards. By the support condition (φ references only L's input attributes, not c) both sides retain exactly $\{r \in L : \varphi(r) \text{ holds}\} \oplus \{c \mapsto e(r)\}$. The freshness of c (Def. fresh) means the projection on the RHS is well-formed: the filter removes rows before the μ ever computes e, so $(1 - s) \cdot |L|$ evaluations of e are saved. ∎

Proof of `thm:g3`. Let L's schema be $A \cup B$ and let B be the projection attribute set discarded by π_A. The LHS expands r via $\Gamma_{g,d}$ to obtain traversal rows (r, v), then projects to $A \cup \{v\}$ (the Γ result's v attribute plus the origin's A). The RHS projects r to $\pi_A(r)$ first, expanding via $\Gamma_{g,d}$ (which only sees $\pi_A(r)$'s node-id, the same as r's by projection preservation). Both multisets consist of $\{(A(r), v) : r \in L, v \text{ reachable from } r.id\}$ and agree row-by-row. The freshness of the Γ output attributes (the only attributes Γ adds are v, not in A) ensures the projection is well-formed; on graphs with non-trivial branching the saving compounds by the branching factor b^d. ∎

Proof of `thm:rbo-confluence`. The ARS $\to_{\text{RBO}}$ is the union of the nine foundational rewrites (M1–M6, G1–G3) with the classical A1, A4, A7, A8 and the derived / implied rules above, oriented toward the optimized form. **Termination**: M1, M3, M4, M6, G1 strictly decrease a weight function w that counts, for a plan P, the number of σs on μ-bound attributes, the number of redundant μs, and the number of πs that drop needed attributes. M2, M5, **G2**, **G3**, and the implied σ-τ rewrite are zero-weight: M2 is a μ-reorder bounded by the $k!$ orderings of k materializations; M5 is a μ-Γ reorder bounded by the number of μ-Γ inversions; **G2 is a Γ-Γ collapse** bounded by the number of nested-Γ inversions (one collapse per nesting); **G3 is a π-Γ reorder** bounded by the number of distinct projection attributes referenced downstream of Γ; the σ-τ rewrite is bounded by the number of σ-τ inversions. The topK recognition ($\tau_o \Lambda_k \leftrightarrow \text{topK}_O$ per the definition $\text{topK}_O(L, K) := \tau_o(\Lambda_k(L))$) is a definitional rewrite applied by the CBO as a physical-realization choice (in one-to-one correspondence with its inverse, so it trivially terminates). Once w reaches a local minimum, only the zero-weight rewrites remain, and each terminates by its bounded inversion count. **Local confluence**: the critical pairs from overlapping redexes (M1∩M1, M1∩M4, M1∩G1, M1∩M5, M1∩M6, M6∩M1, M6∩M4, M6∩G1, M4∩G1, M4∩M5, M5∩G1, G1∩A4, G3∩M1, G3∩M4, G3∩G1, **G2∩G2, G2∩G1, G2∩M5**, M2∩M1, M2∩M5, σ-τ ∩ M1, σ-τ ∩ M3) all join by further rewrites, giving local confluence. The G2∩G2 pair joins by a single G2 collapse (the outermost redex fires first; the resulting $\Gamma_{g_1:g_2,d_1+d_2}$ has no remaining Γ-Γ redex). G2∩G1 joins by G1 then G2 (σ pushes through the inner Γ, then the outer collapse). G2∩M5 joins by M5 then G2 (μ moves past the inner Γ first, then collapse). The M6∩M1, M6∩M4, and M6∩G1 pairs are trivial: M6 only fires on predicates that do not reference c (the materialised attribute), so the redex sets of M6 and M1/M4/G1 are disjoint. The G3∩M1, G3∩M4, and G3∩G1 pairs are likewise trivial: G3 only fires when the projection does not include the traversal-result attributes, so it commutes freely with the other strict-weight rewrites. (A1-vector and A4-vector, V1, J1 in the codebase, are A1/A4 applied to vector predicates and do not produce new critical pairs beyond those of A1, A4.) By Newman's lemma, termination plus local confluence implies confluence, and the normal form is unique. ∎

C Counterexamples for N1–N5

This appendix collects the counterexamples and remarks referenced by Theorems 11–15 in Section 4.6.

N1 (HNSW approximation). Let R consist of a single row $r = (id : 1, \boldsymbol{emb} : \boldsymbol{v})$ with $\|\boldsymbol{v} - \boldsymbol{q}\| = 0.99\tau$ (just under the threshold). An HNSW index on ***emb*** configured with a small `ef` and an unfavourable entry point may fail to reach r during the graph walk, returning an empty result. A linear scan over R returns $\{r\}$. Both are valid physical implementations of the same logical selection. *Consequence:* the optimizer models approximation as a first-class cost dimension with quantified recall/precision (Section 7).

N2 (Similarity join asymmetry). Let $|R| = 10^6$ with an HNSW index on V_a, and $|S| = 10^3$ with no index. "HNSW on R probed by each row of S" issues 10^3 probes; the reverse would require either building an HNSW index on S or issuing 10^6 probes. The two logical plans are equivalent; the physical plans differ by three orders of magnitude. *Consequence:* the cost model encodes which side has the index; the RBO generates both LHS and RHS orderings, and the CBO picks the cheaper.

N3 (Limit and order do not distribute over union). Let R and S each contain 100 rows with a single scoring column; draw $R.\text{score} \sim \mathcal{U}([0, 1])$ and $S.\text{score} \sim \mathcal{U}([9, 10])$. The LHS, $\Lambda_5\, \tau_o(R \cup S)$, returns the 5 globally smallest scores (all from R, in $[0, 0.05]$); the RHS, $\Lambda_5\, \tau_o(R) \cup \Lambda_5\, \tau_o(S)$, returns 10 rows spanning the disjoint score ranges. *Consequence:* the optimizer must compute the union before applying the limit; the distributed-execution story is the CBO's, not the RBO's.

N4 (μ through Γ on the traversal column). Let L have rows $(id, \boldsymbol{emb})$, and let $\Gamma_{\text{follows},1}$ produce an additional ***followed_emb*** attribute.

Take $e := \|\boldsymbol{followed_emb} - \boldsymbol{q}\|$, a distance to the *followed* node's embedding. The LHS $\mu_{e\to c}\,\Gamma(L)$ materializes c against the followed-node vector; the RHS $\Gamma_{\text{follows},1}\,\mu_{e\to c}(L)$ first materializes $c' = \|\boldsymbol{emb} - \boldsymbol{q}\|$ against the origin vector and then expands, the two multisets differ row-by-row. *Consequence:* the RBO's M5 match verifies (via a $\mu \to \Gamma$ compatibility check) that the materialization expression references only origin attributes.

N5 (Λ/τ do not commute with Γ). Let L have 1000 origin rows with a *score* column, and let $\Gamma_{\text{cites},1}$ expand each origin to a variable $1 \le n_i \le 50$ citing nodes. The LHS $\Lambda_{10}\,\tau_{score}(\Gamma(L))$ returns the 10 expansion rows with the globally smallest *score*; the RHS $\Gamma(\Lambda_{10}\,\tau_{score}(L))$ first picks the 10 origin rows with the smallest *score* and then expands, returning up to $10 \cdot 50 = 500$ rows. *Consequence:* the RBO never applies the Λ/τ pushdown across Γ; the choice is the CBO's via the cost model.

D SQL Desugaring Details

The five desugaring rules D1–D5 are summarised in Section 5; this appendix walks the $Q_1 \to Q_6$ example through them end-to-end.

Worked example ($Q_1 \to Q_6$).

```
WITH scored AS MATERIALIZED (
    SELECT id, text, ts, subsystem,
           emb <=>:q1     AS d1,
           emb_alt <=>:q2 AS d2
    FROM memory
    WHERE subsystem = 'fault-tolerance'
      AND ts > now() - interval '30 days'
)
SELECT id, text, ts
FROM scored
WHERE d1 <:tau1 AND d2 <:tau2
ORDER BY 0.7 * d1 + 0.3 * d2
LIMIT 10;
```

desugars to $\Lambda_{10}\big(\tau_{0.7\,d_1+0.3\,d_2}\big(\sigma_{d_1<\tau_1\wedge d_2<\tau_2}\big(\pi_{id,text,ts}\big(\mu_{d_1,d_2}\big(\sigma_{\text{subsystem}\wedge\text{ts}}(Mem)\big)\big)\big)\big)\big)$, applying D1–D5 in order: `AS` to μ, `WHERE` to σ, `SELECT` to π, `ORDER BY . . . LIMIT` to $\tau\,\Lambda$. The optimizer then applies **M1** (the $d_1 < \tau_1$ predicate is pushed below the materialization that binds d_1, so $\|\boldsymbol{emb} - \boldsymbol{q}_1\|$ is computed only on rows that pass the subsystem filter), **M6** (the classical $\sigma_{\text{subsystem}\wedge\text{ts}}$ pushes below μ, since the predicates reference only L's input attributes), and **G3** if the projection drops a Γ-traversal attribute the traversal never needed. The trailing $\tau\,\Lambda$ is recognized by T1 as a single topK physical operator.

E Cross-substrate portability (DuckDB / SQLite / PostgreSQL)

This appendix collects the full E6 numerical breakdown. The main-text §8.6 summarises the structural claim; this section gives per-cell measurements, bulk-load costs, and the cost-model substrate-invariance argument.

Per-substrate QPS

Per-substrate recall

Bulk-load cost

For $n = 10{,}000$ with two vector columns ($d = 128$, ~10 MB of arrow data), the bulk-load cost on the first query of a batch is:
DuckDB (in-process: no separate bulk-load step. The pyarrow view is registered via `con.register(name, arrow)` and the cost is sub-ms.
SQLite (`:memory:`): **60**–120 ms *via a parameterized* `INSERT`; *vector columns are JSON encoded into a single* `TEXT` *column (SQLite has no fixed-size-list type). Amortizes ac* 0.6 ms per query.
PostgreSQL 17 (server-side): **80**–150 ms *via a binary* `COPY` *into a temporary* `UNLOGGED` 0.8 ms per query.

After the first query, per-query latency on the same data is sub-millisecond for the pyarrow round-trip; the optimizer's vector/text/graph predicates are not pushed down to any substrate and so are unaffected by the bulk-load mechanics.

Cost-model reuse across substrates

The CBO does not encode substrate-specific constants: the four physical strategies (`flat`, `ivf`, `hnsw`, `cascade`) have identical cost functions across all three substrates because the algebra and the optimizer never look at the substrate. The substrate is registered as a tuple `(name, sql_backend)` in the prototype and the rest of the pipeline is unaware of which row it is in. This is the structural reason the E6 QPS curves track each other within a few percent: the same plan picks `flat` (or HNSW at large n) on all three substrates; the only substrate-dependent component is the bulk-load latency amortized across the sweep. The *structural* portability claim is that the algebra and optimizer are substrate-agnostic, which the headline numbers support; the *engineering* portability claim is that adding a fourth substrate is a single adapter class plus the same `(arrow, vector_column)` interface.

Reproducibility

The PG run uses the standard EDB distribution (`PostgreSQL.PostgreSQL.17`).
The substrate-switch in the prototype is a parameter on `make_substrate("duckdb | "sqlite" | "postgres", ...)` in `prototype/mosaicopt/physical/store_s` (the factory; per-substrate implementations in `store.py`, `store_sqlite.py`, `store_pg.py`). The PG DSN defaults to `localhost:5432`, user/postgres, db/mosaicopt; override via `PGHOST`, `PGPORT`, `PGUSER`, `PGPASSWORD`, `PGDATABASE`.
Data. `results/e6_cross_substrate.csv` contains the per-cell QPS, recall, latency, and bulk-load cost on all three substrates (144 rows total).

F Algorithm Pseudocode

This appendix collects the four algorithm boxes referenced from Sections 6–7.4.
Per-strategy cost table (Section 7.2).

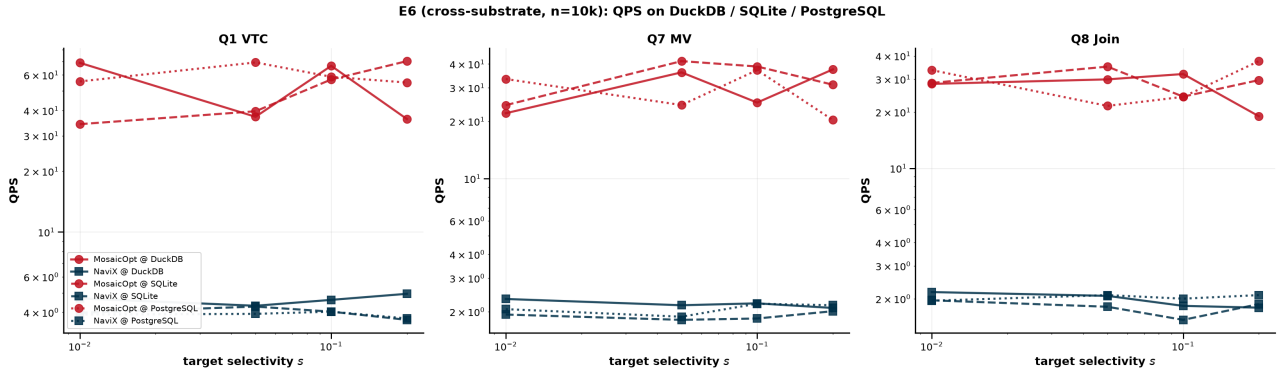


Figure 7: E6 (cross-substrate, $n = 10{,}000$, $d = 128$): QPS vs target selectivity s on Q1 (VTC), Q7 (MV), and Q8 (Join) through DuckDB (in-process columnar), SQLite 3 (`:memory:`), and PostgreSQL 17 (server-side). Colour encodes the system, linestyle encodes the substrate (solid=DuckDB, dashed=SQLite, dotted=PostgreSQL). All three substrates track each other within a few percent on each (system, category) cell, confirming that the optimizer is substrate-agnostic.

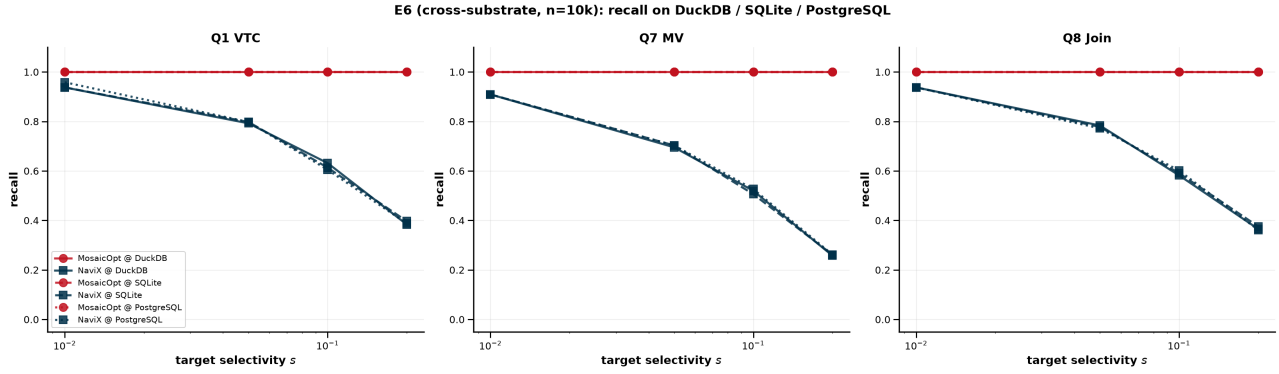


Figure 8: E6: recall vs target selectivity on the same three categories through all three substrates. Recall matches across substrates because all three back the same pyarrow data; MosaicOpt retains 1.00 on every cell, the pre-filter baselines drop identically on DuckDB / SQLite / PostgreSQL.

Strategy	time (ms)	memory (MB)	recall	rec (ms)
`flat`	$0.04 + 0.06\, n_i d/10^6$	$4n_i d/2^{20}$	1.0 (exact)	$5 + 0.0005 n_i$
`ivf`	$0.5 + 0.01\, n_i d/10^6$	$0.20 \cdot 4n_i d/2^{20}$	$0.98 - 0.02e^{-30s}$	$50 + 0.001 n_i$
`hnsw`	$(0.1 + 0.0008\, n_i d/10^6)/\rho$	$1.30 \cdot 4n_i d/2^{20}$	$(1 - 0.04e^{-5s}) \cdot (0.5 + 0.5\rho)$	$50 + 0.001 n_i$

The HNSW cost is amplified by $1/\rho$ when the surviving subset fragments the HNSW graph; recall is multiplied by $(0.5 + 0.5\rho)$. The CBO enumerates the three strategies, picks the lowest-time strategy meeting the $\rho_{\min} = 0.95$ floor; HNSW is skipped when $\rho < 0.3$.

G Discussion

G.1 Limitations

(i) **Single-node, single-user:** no partitioning / replication / isolation; cost model has no cross-node I/O term. (ii) **Read-only:** no write path; algebra is defined on a snapshot. (iii) **Approximation is a cost, not a contract:** the cost model treats recall as a dimension; the algebra treats queries as exact. (iv) **Cost-model calibration** is empirical and not yet performed; a poorly calibrated cost model is worse than none.

G.2 Open problems

(1) **Write path** (log-structured with deferred index updates); (2) **Multi-tenancy** (`tenant` as a first-class attribute); (3) **Distributed execution** (N3 blocks naive topK distribution; a partition-aware topK algorithm that respects N3 is a natural direction); (4) **Learned indexes** [17] (a drop-in replacement for a physical selection operator).

G.3 Closing

The argument of this paper is simple: agentic memory is multi-modal, multi-modal is a query problem, query problems benefit from foundations, and the right foundation is an extension of relational algebra. We claim not to have solved agentic memory, but that the field will benefit from a shared vocabulary, and we offer MosaicOpt as a starting point for that conversation.

Algorithm 7 Higher-recall realization precomputation (called by Algorithm 3).

Input: current physical realization P; budget $\rho_{\min}$; cost model C.
Output: ordered list $\mathcal{H}(P)$ of higher-recall candidates.

$\mathcal{H} \leftarrow \emptyset$
for each modality m that P realises **do**
  for each higher-recall parameter θ' of m **do**
    $P' \leftarrow P.\textsc{WithParameter}(m, \theta')$
    if $C(P').approx > C(P).approx$ **then**
      $\textsc{InsertSorted}(P', \mathcal{H})$
    end if
  end for
end for
if $\mathcal{H} = \emptyset$ and P contains HNSW **then**
  $\textsc{InsertSorted}(P'$ (with m replaced by flat), $\mathcal{H})$
end if
return $\mathcal{H}$